

# Forest LiDAR Point Cloud Processing

May 12, 2026

## 1 Hochschule Neubrandenburg

### 1.0.1 University of Applied Sciences

---

### 1.1 Integrating Large Language Models into Geospatial Workflows:

#### 1.1.1 A Case Study in Forest LiDAR Point Cloud Processing

---

---

|                              |                           |
|------------------------------|---------------------------|
| <b>Author:</b>               | Nazmus Sadat              |
| <b>Matriculation Number:</b> | 2826008                   |
| <b>Email:</b>                | lg26006@hs-nb.de          |
| <b>Supervisor:</b>           | Prof. Dr. Tobias Hillmann |
| <b>Date of Submission:</b>   | June, 2026                |

---

#### 1.1.2 Abstract

Forests play a critical role in global carbon sequestration, biodiversity conservation, and timber production, making accurate structural assessment a priority in modern forestry. Light Detection and Ranging (LiDAR) technology has emerged as one of the most powerful tools for capturing high-resolution, three-dimensional forest structure data. However, processing the resulting large-scale point clouds poses significant computational challenges, particularly in memory-constrained environments. This project presents a complete, Python-based pipeline for the analysis of a large-scale terrestrial LiDAR dataset — *forest.las* — comprising 178,297,784 points across a  $90\text{ m} \times 90\text{ m}$  forest plot, developed with iterative assistance from a Large Language Model (LLM).

The central technical challenge was processing a 4.3 GB LAS file on a standard laptop without memory overflow. This was addressed through a chunked reading strategy using the *laspy* library's `chunk_iterator()` method, combined with a dictionary-based gridding approach using Python's `defaultdict` to build a Canopy Height Model (CHM) at 2-meter spatial resolution. The resulting CHM revealed a maximum canopy height of 27.16 m and a mean canopy height of 12.42 m, consistent with values reported for mature temperate forest stands. Complementary visualizations were produced, including a 2D top-down scatter plot derived from a 1% point sample and an interactive 3D point cloud rendered from approximately 595,000 points, collectively providing a multi-perspective view of the forest's vertical structure.

Building on the CHM, an individual tree detection pipeline was implemented using local maxima filtering, identifying up to 108 candidate tree apex positions. For each detected tree, trunk-zone points were extracted using a radial search and height-based filtering, followed by Principal Component Analysis (PCA) to estimate the dominant trunk axis orientation. The inclination angle between each fitted trunk axis and the global vertical was computed and used to classify trees as vertical (deviation below  $10^\circ$ ) or leaning. An iterative diagnostic procedure was required to correctly calibrate the absolute Z-coordinate window for trunk extraction, revealing that the effective ground surface in this dataset lies near  $Z = 0.5$  m rather than at the global Z minimum.

Throughout the development process, Grok 4.3 by xAI served as the primary LLM collaborator. Following a systematic comparison of leading models — including Claude Opus 4, GPT-5.5, Gemini 3.1 Pro, and DeepSeek V4 — Grok was selected for its practical, solution-oriented approach to memory optimization, strong iterative debugging capability, and favorable cost-accessibility balance for academic use. The LLM significantly accelerated development by suggesting memory-efficient algorithms, resolving runtime errors, and supporting the generation of scientific documentation. However, the project also demonstrated that LLM assistance requires continuous human oversight: the initial Z-window parameterization, though logically plausible, was empirically incorrect and required developer-led diagnostic validation to resolve.

The final verticality classification identified 3 of 20 analyzed trees as having nearly vertical trunks, with a mean deviation angle of  $41.12^\circ$ . The high proportion of leaning classifications is attributed primarily to methodological artifacts — trunk point contamination from neighboring vegetation and false apex detections at the scan periphery — rather than genuine ecological trunk inclination. Future improvements include proper ground normalization, marker-controlled watershed segmentation, statistical outlier filtering, and ground-truth validation. This project demonstrates that a rigorous, end-to-end LiDAR analysis workflow, encompassing data loading, CHM generation, individual tree detection, and PCA-based structural classification, is achievable on standard hardware using exclusively open-source Python libraries, and that LLM-assisted geospatial development is a practically viable and academically valuable approach when conducted with appropriate critical rigor.

## 1.2 1. Introduction

Forests are an important component of the terrestrial ecosystem due to their high biodiversity, complex structure and multiple uses. The number of studies on monitoring carbon emissions from forests with remote sensing techniques has increased significantly in recent years (Campbell et al., 2012). Numerous environmental services and socio-economic benefits are derived from forest ecosystems, including carbon sequestration, wildlife habitat, recreational opportunities, and the supply of wood and non-wood products (Gómez et al., 2019). In contemporary forestry, the accurate assessment of forest structure is of paramount importance—particularly when it comes to identifying trees with nearly vertical and straight stems (Chen et al., 2018; Tran et al., 2023). The stems are generally vertically oriented, which is correlated to higher timber quality, improved structural stability and more accurate allometric equations for biomass estimates (Fan et al., 2025). Light Detection and Ranging (LiDAR) technology has become one of the most important tools in modern geomatics, forestry, and environmental monitoring (Aguilar et al., 2024). It can collect important data for forest ecology, such as forest vertical stratification, gas exchange, transpiration, and canopy carbon content (Koetz et al. 2007). It also helps in forest accounting, such as the location of the tree within a plot, measuring tree height, crown measurements, and volume estimation (Maltamo et al. 2005). Airborne or terrestrial LiDAR systems generate dense three-dimensional point clouds

that represent the Earth’s surface and vegetation with high accuracy (Dassot et al., 2011). These datasets are particularly valuable for forest analysis because they allow the derivation of critical structural parameters such as tree height, canopy density, biomass estimation, and gap detection (Dubayah et al. 2010; Qi et al. 2019).

The dataset provided for this project, `forest.las`, is a typical example of such data. It contains 178,297,784 points and occupies 4.3 GB of storage. The scanned area is relatively small (approximately 90 m  $\times$  90 m), yet the sheer volume of points creates significant computational challenges. The height values range from approximately -7.13 m to +27.16 m, with a maximum canopy height of 27.16 m and an average canopy height of around 12.42 m (derived at 2 m grid resolution). Processing such large point clouds requires careful memory management, efficient algorithms, and robust visualization techniques.

Challenges in LiDAR Data Processing; Handling large LiDAR datasets poses several technical difficulties:

- Memory constraints: Loading the entire file into RAM is often impossible on standard laptops or even mid-range workstations.
- Computational efficiency: Operations such as gridding, height normalization, and visualization become extremely slow without down sampling or chunked processing.
- Visualization demands: Effective communication of results requires both 2D raster representations (e.g., Canopy Height Models) and interactive 3D views.
- Data volume vs. insight: Extracting meaningful forest metrics from millions of points while maintaining accuracy is non-trivial.

Traditional GIS software (e.g., QGIS, ArcGIS) and specialized libraries (e.g., `laspy`, `lidR`, `PDAL`) are commonly used, but they often demand significant programming expertise and careful workflow design when dealing with big data (Espejo et al. 2020).

Role of Large Language Models (LLMs) in Geospatial Tasks Autonomous agents have demonstrated great promise in a wide range of applications, from robotics and healthcare to finance and transportation (Russell and Norvig 2022). They have many benefits such as better productivity, less errors and the ability to do difficult tasks that a human would find hard or time-consuming to do (Stone et al. 2022). In recent years, Large Language Models have emerged as powerful assistants in programming and data analysis tasks, including geospatial applications (Li et al. 2023). LLMs can generate code snippets, debug errors, suggest efficient algorithms, and help document workflows in natural language (Brown et al. 2020). Studies and practical implementations (e.g., GeoAgent systems, ChatGeoAI, and integrations with PyQGIS) demonstrate that LLMs can lower the barrier for non-experts and accelerate development in geomatics projects (Gao, 2020).

This final project leverages an LLM (Grok by xAI) iteratively throughout the development process — from initial code suggestions and memory-efficient chunk reading strategies to debugging, visualization improvements, and comprehensive documentation. The combination of Python programming in Jupyter Notebook with LLM assistance exemplifies the aspects of the project: interactive data exploration, professional visualization, and clear scientific documentation.

Project Objectives; The main objectives of this work are: - Develop a memory-efficient pipeline to load and process the large `forest.las` file without crashing due to memory limitations. - Generate a Canopy Height Model (CHM) at 2 m resolution, which serves as a fundamental raster product for forest structure analysis. - Create informative 2D top-down visualizations and an interactive 3D point cloud view to support visual interpretation. - Extract and present key forest metrics

(maximum height, average height, etc.). - Canopy Height Model generation at 2 m resolution - Individual tree detection using local maxima filtering, - Trunk point extraction via radial search, - Principal Component Analysis (PCA) for trunk axis fitting, - Quantitative verticality assessment and classification. - Document the entire workflow thoroughly, highlighting both technical challenges and the supportive role of LLMs. - Reflect on the integration of AI tools in geospatial programming tasks as part of the academic module learning outcomes.

This project demonstrates the integration of large-scale LiDAR data processing with advanced geometric analysis, supported by LLM assistance for algorithm design, memory optimization, and iterative debugging. The developed workflow not only addresses the technical challenges of handling big geospatial data but also delivers actionable forestry insights through professional visualizations and statistical evaluation.

The remainder of this report is structured as follows: Section 2 compares different Large Language Models with respect to their suitability for coding and geospatial tasks. Section 3 justifies the prioritization of a specific LLM. Section 4 details the programming task and implementation. Finally, Section 5 evaluates the results and discusses limitations and future improvements.

## **1.3 2. Comparison of LLMs**

LLMs have become essential tools for programming and data analysis tasks (Huang et al. 2024). This section compares leading LLMs available in 2026 with a specific focus on two critical aspects for this project: coding/programming capabilities and handling large data / geospatial tasks.

### **2.1 Coding / Programming Capabilities:**

Coding performance is typically evaluated using benchmarks such as SWE-Bench Verified (real GitHub issue resolution), LiveCodeBench, Terminal-Bench, and HumanEval (Cohn et al. 2010). As of early 2026, the landscape is highly competitive:

The competitive landscape for code generation and agentic software engineering is led by Anthropic’s Claude Opus 4.7 / 4.6. Ranking first overall, it achieves an approximate score of 83–87% on the rigorous SWE-Bench Verified benchmark. Claude’s key strengths lie in its more neutral complex reasoning, multi-file refactoring, and long-horizon agentic coding capabilities. However, its adoption is somewhat constrained by strict rate limits and a relatively high pricing structure, ranging from 5 dollar to 25 dollars per million tokens, operating within a context window of 200K to 1M tokens (Yang et al. 2025).

Close behind in the second position is OpenAI’s GPT-5.5 / 5.4, which scores approximately 75–82% on SWE-Bench Verified. This model excels in agentic workflows, fast tool execution, speed, and benefits from a highly mature developer ecosystem. Its primary weakness is that it remains slightly behind Claude in deep reasoning tasks. It offers a 128K to 1M token context window, with competitive API pricing estimated between 2.50 dollars and 15 dollars per million tokens (Agarwal et al. 2025; Yang et al. 2025).

In third place, Google’s Gemini 3.1 / 2.5 Pro delivers a strong SWE-Bench Verified performance of 75–80%. Gemini’s standout advantages include powerful multimodal integration, cost-effectiveness, and an evidence-based context window spanning up to 1M to 2M tokens. While it is highly scalable, it occasionally displays less precision in specific code formatting and style guidelines. Its pricing is highly competitive, positioned at 2 dollars to 12 dollars per million tokens (Team et al. 2023; Yang et al. 2025).

xAI’s Grok 4.1 / 4.3 ranks fourth, achieving an approximate SWE-Bench Verified score of 72–78%. Grok is characterized by strong real-time reasoning, helpful iterative debugging features, and high-quality documentation generation, though it is currently limited by a smaller supporting developer ecosystem compared to the top three leaders. It supports a context window of 128K+ tokens at highly competitive pricing rates (Yang et al. 2025; Ma et al. 2026).

Rounding out the top five is DeepSeek’s open-source DeepSeek V3.2 / V4, which scores an impressive 72–80% on the SWE-Bench Verified evaluation. As an open-source model, it represents an outstandingly cost-effective solution with a very low price-per-token ratio and a large context window, though its primary drawback remains a less polished user and API interface (Zhu et al. 2024; Yang et al. 2025).

Analysis of Coding Capabilities: - Claude (Opus/Sonnet series) consistently leads in complex, real-world software engineering tasks. It excels at understanding ambiguous requirements, maintaining consistency across large codebases, and producing clean, well-documented code — skills that were heavily used during iterative development of this project (Yang et al. 2025). - GPT-5.5 series shines in agentic workflows (using tools, running code, debugging loops) and benefits from the most mature developer ecosystem (Cursor, VS Code extensions, etc.) (Agarwal et al. 2025; Yang et al. 2025). - Gemini offers excellent value and very large context windows, making it suitable for projects involving extensive documentation or multiple files (Team et al. 2023; Yang et al. 2025). - Grok performed very well in iterative, conversational debugging and memory-efficient algorithm suggestions, with a helpful, direct communication style (Yang et al. 2025; Ma et al. 2026).

## 2.2 Handling Large Data / Geospatial Tasks

According to Li et al. (2018); Processing large geospatial datasets like LiDAR point clouds (178 million points, 4.3 GB) requires specific capabilities: - Long context understanding — to handle code that processes big files chunk-by-chunk. - Memory management reasoning — suggesting efficient algorithms (chunking, downsampling, dictionaries instead of full DataFrames). - Geospatial domain knowledge — understanding `.las` format, Canopy Height Models, coordinate systems, etc. - Numerical & scientific computing — working with NumPy, pandas, laspy, matplotlib, and avoiding memory errors.

Performance Comparison for Large Data / Geospatial Tasks: - Claude Opus 4.x: Excellent at suggesting sophisticated memory-efficient strategies and understanding geospatial concepts. Strong in writing clear scientific documentation (Yang et al. 2025). - Gemini 2.5/3.1 Pro: Very strong due to its massive context window (1M+ tokens), making it ideal for analyzing long code outputs or entire notebooks at once (Team et al. 2023; Yang et al. 2025). - GPT-5.5: Very good at tool calling and iterative problem-solving (e.g., “fix this MemoryError” loops) (Agarwal et al. 2025; Yang et al. 2025). - Grok 4: Particularly helpful in real-time, conversational problem-solving for memory issues. It provided practical, step-by-step guidance for chunked processing of the 4.3 GB `.las` file without loading everything into RAM — exactly what was needed for this project (Yang et al. 2025; Ma et al. 2026). - DeepSeek models: Strong open-source alternative for local deployment when dealing with sensitive geospatial data (Zhu et al. 2024; Yang et al. 2025).

## 2.3 Context Window Size:

Context window size is crucial when working with long code, large error outputs, and entire notebooks. At the forefront of context capacity is Google’s Gemini 3.1 Pro, which leads the landscape with an expansive 1M to 2M token context window. This massive capacity introduces the unique structural advantage of allowing developers to ingest entire Jupyter notebooks, complete codebase

repositories, or deep historical logs in a single operational turn. However, this vast retention capacity comes with a trade-off, as Gemini occasionally exhibits less precision in strict code style conformity or formatting adherence when processing maximum-capacity inputs. Following this is Anthropic’s flagship hybrid-reasoning model, Claude Opus 4 (specifically including the 4.7 and 4.6 iterations), which supports a context window ranging from 200K up to 1M tokens. Claude Opus is highly optimized for sustained multi-turn conversations and thorough, multi-file code reviews, retaining high retrieval precision even near its upper token limits. The primary constraint of deploying Opus for extreme long-context tasks is its premium pricing structure, which can make sustained, maximum-window developer sessions highly expensive. Representing a highly balanced alternative, OpenAI’s GPT-5.5 features a context window spanning 128K to 1M tokens. This capacity provides a highly efficient equilibrium between execution speed and overall context capacity, though its maximum window remains smaller than Gemini’s. Similarly, xAI’s Grok 4.3 offers a 128K+ token context window, which proves entirely sufficient for localized, iterative debugging workflows, though it is not designed to match the industry’s largest data-ingestion models. Finally, open-source models like DeepSeek V4 boast very large, scalable context capacities that are exceptionally suited for localized, self-hosted deployments. While highly cost-effective and secure for private codebases, the overall efficiency and retrieval accuracy of DeepSeek. (Team et al. 2023; Agarwal et al. 2025; Yang et al. 2025; Ma et al. 2026). Relevance to Project: A large context window helped significantly when reviewing long error traces and previous code versions during memory optimization.

## **2.4 Reasoning Quality:**

According to Yang et al. (2025); Reasoning quality determines how well an LLM understands complex problems and breaks them down logically. - Claude Opus 4: Currently considered the best in step-by-step logical reasoning and avoiding hallucinations in technical tasks. - Grok 4.3: Strong real-time reasoning with a practical, solution-oriented approach. It performed very well in diagnosing MemoryError issues and suggesting efficient alternatives (e.g., using `defaultdict` instead of `pandas` for 178M points). - GPT-5.5: Very good chain-of-thought reasoning and creative problem solving. - Gemini 3.1: Solid reasoning with strong multimodal understanding.

## **2.5 Cost / Accessibility:**

Cost and accessibility are important factors when working on academic projects with limited budget. Claude Opus 4 is positioned at a high-cost level, accessible via a 20 dollar per month web-based Pro subscription or a usage-based API. Because of its premium pricing, it is best reserved for professional, production-ready enterprise applications and high-quality software engineering tasks where deep reasoning overrides budget constraints. Similarly, OpenAI’s GPT-5.5 sits at a medium-to-high-cost tier, offering broad accessibility through a standard ChatGPT Plus subscription (20 dollar/month) alongside a pay-per-use developer API. This accessible and highly mature ecosystem makes GPT-5.5 the premier industry choice for general developer utility and rapid, iterative software prototyping. Conversely, more economical options provide excellent utility for budget-sensitive workflows and research. Google’s Gemini 3.1 Pro offers exceptional accessibility, featuring a highly robust free tier alongside Google AI Studio. This low-cost barrier to entry makes Gemini 3.1 Pro particularly appealing to students, independent researchers, and developers tackling massive, long-context codebase analysis. Similarly, xAI’s Grok 4.3 utilizes a hybrid model, offering limited free-tier access alongside standard xAI platform subscriptions. Characterized by a low-to-medium cost level, Grok 4.3 is highly suited for academic environments and fast, iterative code development. Lastly, DeepSeek’s open-weights model, DeepSeek V4, operates at a very low-cost level, distributed primarily through open-source weights for local hosting or via highly discounted API endpoints.

This framework makes DeepSeek V4 the ideal deployment choice for cost-sensitive projects and enterprise architectures that mandate strict data privacy and local infrastructure control (Team et al. 2023; Agarwal et al. 2025; Yang et al. 2025; Ma et al. 2026).

Analysis: For a student project like this, Grok and Gemini offer the best cost-accessibility balance. Grok provided unlimited helpful conversation during the development of this project without high costs.

## 2.6 Strengths & Weaknesses for LiDAR Processing Task:

This project required memory-efficient processing of a 4.3 GB LAS file with 178 million points, chunked reading, dictionary-based gridding, and visualization. When selecting an LLM specifically for heavy LiDAR point cloud processing workflows, the models exhibit distinct architectural and practical trade-offs. Representing the highest overall suitability for this task is xAI’s Grok 4.3. Grok 4.3 excels in providing highly practical memory solutions and displays a patient, effective approach to iterative debugging. It is particularly adept at suggesting efficient out-of-core chunk processing routines and optimizing Python’s memory footprint using structures like defaulted over heavy pandas Data Frames. Its primary limitation is a slightly less formal, more conversational academic writing style, though its highly robust code execution and debugging capabilities make it an outstanding candidate for hands-on engineering. Ranking with high suitability are Anthropic’s Claude Opus 4 (including 4.7 / 4.6) and OpenAI’s GPT-5.5. Claude Opus 4 delivers the less hype wording overall code quality and produces exceptionally clear, academically rigorous documentation. It is highly capable of reasoning through complex spatial indexing and geometric algorithms. However, its premium API pricing makes long-context debug sessions expensive, and it can occasionally exhibit over-cautiousness, sometimes refusing to run or generate certain complex local scripts. Meanwhile, GPT-5.5 is highly effective for rapid prototyping and fast debugging iteration, showing a strong ability to isolate and patch runtime execution errors quickly. Its weakness lies in a tendency to occasionally suggest standard, memory-heavy array algorithms that fail to account for the massive scale of big geospatial data, requiring closer supervision from the developer. Finally, Google’s Gemini 3.1 Pro offers good suitability for LiDAR tasks. Gemini’s core advantage is its massive context window, which allows developers to feed entire, multi-step Jupyter notebooks into a single prompt for holistic review. However, it can occasionally lack precision with low-level, granular memory optimization techniques, sometimes requiring additional prompting to avoid out-of-memory (OOM) errors when processing dense .las files (Team et al. 2023; Agarwal et al. 2025; Yang et al. 2025; Ma et al. 2026).

Key Observation: Grok excelled in the specific challenges of this project — repeatedly helping to solve `MemoryError` issues, suggesting efficient chunked reading with `laspy.open()`, and guiding the use of `defaultdict` for the Canopy Height Model. This practical, solution-focused support was more valuable than theoretical benchmark leadership for this real-world large-scale geospatial task (Ma et al. 2026).

Key Insight: No single LLM is perfect for every task. The best approach (as demonstrated in this project) is iterative collaboration with one primary LLM while occasionally cross-checking with others (Dhanuka 2025). This hybrid human-LLM workflow significantly accelerated development and reduced debugging time (Passerini et al. 2025).

Summary of LLM Comparison: After evaluating the models across coding ability, large data handling, context size, reasoning quality, cost, and domain-specific suitability for LiDAR processing, Grok 4.3 by xAI was selected as the primary LLM assistant for this final project. It offered the best balance of practical problem-solving capability, accessibility for students, and effective support for

memory-constrained geospatial data processing (Yang et al. 2025; Ma et al. 2026).

Overall Assessment: For this specific project involving memory-constrained geospatial data processing, a combination of strong reasoning and practical coding suggestions was more valuable than raw benchmark scores.

## 1.4 3. Prioritization of an LLM

### 3.1 Selected LLM: Grok 4.3 by xAI

For this final project in the Multimedia module, Grok 4.3 developed by xAI was chosen as the primary Large Language Model to support the entire workflow. From the very beginning — when facing the challenge of loading a 4.3 GB LiDAR file containing 178 million points — until the final documentation and report writing, Grok served as a consistent and reliable programming partner. The decision to use Grok was made after careful consideration of multiple leading models. Throughout the project, all major code development, error debugging, algorithm optimization, and academic writing were carried out through continuous interaction with Grok. This close collaboration proved to be highly effective, especially when dealing with memory constraints and the technical difficulties of processing such a large point cloud dataset (Mohammed et al. 2025). The model not only generated code but also explained the reasoning behind each suggestion, helped refine ideas, and maintained context across long development sessions, which was essential for completing this technically demanding task (Wangsa et al. 2024).

**3.2 Justification with Criteria** The selection of Grok 4.3 was based on a systematic evaluation using the following key criteria, which were especially relevant to the challenges of processing a large 4.3 GB LiDAR point cloud.

1. **Coding Strength and Practical Problem-Solving Ability:** Grok demonstrated excellent coding capabilities throughout the project. It consistently provided clean, functional, and memory-efficient Python code (Samet, 2025). When initial attempts to load the full `forest.las` file failed due to `MemoryError`, Grok quickly suggested using `laspy.open()` with `chunk_iterator()`, `defaultdict` for gridding, and `np.asarray()` with `float32` data types. These practical solutions were more immediately useful than more theoretical answers from other models.
2. **Helpfulness in Iterative Development** One of the biggest strengths of Grok was its effectiveness in iterative, conversational debugging. The development process involved dozens of back-and-forth interactions: testing code, encountering `MemoryError` or `SyntaxError`, and receiving targeted fixes (DataStudios, 2025). Grok maintained context well across multiple turns, remembered previous solutions, and offered progressively better versions of the code. This iterative workflow significantly reduced development time (Wandb, 2025).
3. **Documentation Quality and Educational Value:** Grok provided exceptionally clear explanations alongside code. Every code block was accompanied by detailed comments and reasoning, which greatly helped in creating the high-quality documentation required for this assessment. The model also suggested proper academic structure and helped improve the clarity of explanations in this report (McGee, 2025).
4. **Memory Management and Large Data Handling:** Given the specific challenge of this project (178 million points, 4.3 GB file), Grok’s ability to suggest efficient chunked processing strategies was decisive. It successfully guided the imple-



mentation of a memory-friendly pipeline that processed the entire dataset without loading it all into RAM at once (DataStudios, 2025).

5. **Cost and Accessibility for Students:** As a student project with limited budget, Grok offered an excellent balance of performance and accessibility. It provided high-quality assistance without requiring expensive subscriptions, making it highly suitable for academic work (Yang et al. 2025; Ma et al. 2026).
6. **Reasoning Quality and Patience:** Grok showed strong step-by-step reasoning and remained patient when multiple attempts were needed to solve complex memory issues. It avoided overly generic answers and focused on the specific constraints of the user’s environment (standard laptop with limited RAM) (DataStudios, 2025).

### 3.3 Comparison with Alternative LLMs:

While several powerful LLMs were considered for this project, Grok 4.3 was ultimately prioritized due to its superior performance in the specific context of this assignment. Although Claude Opus 4 is currently regarded as one of the strongest models in terms of code quality and structured reasoning, it tends to be more expensive and sometimes overly cautious, occasionally refusing to provide certain types of code or requiring more refined prompting. In contrast, Grok offered a more practical and solution-oriented approach that was better suited for the iterative trial-and-error process required when handling memory-intensive geospatial data (Yang et al. 2025).

GPT-5.5 is also excellent and benefits from a very mature ecosystem; however, during the critical moments of solving repeated `MemoryError` exceptions while processing the 178-million-point dataset, Grok provided faster and more directly applicable suggestions, particularly regarding chunked file reading and memory-efficient data structures like `defaultdict` (Team et al. 2023; Yang et al. 2025).

Gemini 3.1 Pro stands out with its massive context window, which is advantageous for reviewing entire notebooks, but it occasionally lacked the depth of practical optimization techniques that Grok consistently delivered. Models like DeepSeek V4 offer great value for money, but they showed less familiarity with niche geospatial formats such as `.las` files and required more manual correction (Team et al. 2023; Yang et al. 2025).

In summary, Grok 4.3 struck the best balance between strong technical coding ability, excellent iterative debugging support, clear educational explanations, reasonable cost for a student project, and — most importantly — consistent helpfulness when facing real-world memory and performance challenges (Yang et al. 2025; Ma et al. 2026). This made it the most suitable choice for successfully completing the LiDAR analysis project under the given constraints. The choice of Grok 4.3 proved to be highly effective. The model acted not only as a code generator but as a true programming partner, enabling the successful completion of a technically demanding project that would have been significantly more difficult without LLM assistance.

## 1.5 4. Programming Task

### 4.1 Description of the Task

The core programming task of this final project was to develop a complete, efficient, and well-documented Python workflow capable of processing a large-scale LiDAR point cloud dataset named `forest.las`. The implementation was developed using Python within a Jupyter Notebook environment. Jupyter Notebook was selected because it enables interactive execution of code, visualization

of intermediate results, and efficient experimentation with different point-cloud processing methods. This file, with a size of 4.3 GB and containing 178,297,784 individual points, represents a significant big data challenge commonly encountered in modern geomatics and forestry applications. The notebook-based workflow also simplified debugging and parameter tuning during the analysis of the forest LiDAR dataset. The following Python libraries were used throughout the implementation: `laspy` Reading and processing LAS point-cloud files, `NumPy` Numerical operations and array handling, `Matplotlib` Data visualization, `Scikit-learn` DBSCAN clustering and PCA analysis, `SciPy` Spatial filtering and image processing.

1. Efficient Processing of Large. las File: The most critical challenge was to read and process the massive LiDAR file without causing memory overflow (`MemoryError`), which commonly occurs when attempting to load the entire dataset into RAM at once. The solution required implementing a memory-efficient pipeline using chunked reading techniques (Muñoz et al. 2024). This involved using the `laspy` library’s `chunk_iterator()` method to process the file in smaller portions (e.g., 500,000 points at a time), thereby keeping memory usage within the limits of laptop (Tessl.io, 2025).
2. Creation of a Canopy Height Model (CHM):  
A Canopy Height Model is one of the most important raster products derived from forest LiDAR data. It represents the height of the vegetation canopy above the ground level for each grid cell (Coops et al. 2021). For this project, a CHM was generated at 2.0-meter spatial resolution using a dictionary-based gridding approach (`defaultdict`) to calculate the maximum height (Z-value) in each cell. According to Dhannya et al. (2025), This method was chosen because traditional approaches using `pandas` DataFrames or full `NumPy` arrays caused memory issues with 178 million points.
3. Visualization of Results:  
Effective multimedia presentation of the results was essential for this module. The task included:
  - Creating a high-quality 2D top-down view of the point cloud colored by height.
  - Generating a raster Canopy Height Model visualization using `matplotlib`.
  - Producing an interactive 3D point cloud visualization (rotatable with mouse) to allow better exploration of the forest structure.
4. Tree Apex Detection: After generating the CHM, local maxima detection was used to identify potential tree apex positions. Local maxima represent the highest points within a neighborhood region and are commonly used in forestry applications to estimate tree locations. The apex detection stage enabled approximate localization of individual trees within the forest environment. These detected tree positions later served as reference points for trunk extraction and PCA-based orientation analysis. However, dense canopy overlaps occasionally caused neighboring trees to merge into a single region, creating segmentation challenges in highly populated forest areas.
5. Tree Segmentation Using Spatial Filtering: To isolate individual trees, spatial filtering techniques were applied around detected tree apex locations. Instead of processing the entire forest simultaneously, smaller local regions were extracted around candidate tree positions. This reduced the influence of neighbouring trees and simplified trunk analysis. Additionally, DBSCAN clustering was tested to separate tree structures automatically. DBSCAN groups points based on spatial density and is widely used for point-cloud segmentation. The clustering algorithm was implemented using the following parameters: Initial experiments showed

that large epsilon values caused multiple neighbouring trees to merge into a single cluster. Reducing the neighbourhood radius improved segmentation quality but also increased sensitivity to sparse point distributions. These observations demonstrated the importance of parameter tuning in dense forest environments.

6. **Trunk Region Extraction:** After local tree regions had been isolated, trunk points were extracted using height-based filtering. The lower section of each tree was assumed to contain the trunk structure, while upper regions mainly represented canopy and branch points. To isolate the trunk region, only points within a limited height interval above the local minimum height were retained. The filtering process was implemented as follows: This approach removed most canopy points while preserving the dominant trunk geometry. Although the filtering process improved trunk isolation, some residual noise and neighbouring vegetation occasionally remained in the extracted trunk regions.
7. **PCA for Trunk Orientation:** PCA was used to estimate the dominant orientation of each tree trunk. PCA identifies the direction of maximum variance within a point distribution. When applied to trunk points, the first principal component approximates the main trunk axis. The PCA model was implemented using Scikit-learn: The resulting direction vector represented the estimated orientation of the tree trunk in 3D space. For nearly vertical trees, the Z-component of the principal direction vector approaches 1.0, while leaning trees exhibit larger horizontal components.
8. **Verticality Angle Calculation:** To determine whether a tree trunk was vertically aligned, the angle between the estimated trunk direction vector and the global vertical axis was calculated. The inclination angle was computed using the dot-product relationship:

$$\theta = \cos^{-1} \left( \frac{\vec{v} \cdot \vec{z}}{\|\vec{v}\| \|\vec{z}\|} \right)$$

Trees with small inclination angles were classified as nearly vertical, while larger angles indicated leaning structures.

9. **Tree Classification:** Based on the calculated inclination angle, trees were classified into three categories:

| Angle Range          | Classification  |
|----------------------|-----------------|
| $< 5^\circ$          | Highly Vertical |
| $5^\circ - 10^\circ$ | Nearly Vertical |
| $> 10^\circ$         | Leaning Tree    |

This threshold-based classification enabled automatic identification of trees with almost vertical trunks. The final analysis identified multiple trees exhibiting strong vertical alignment, while others showed noticeable leaning behavior due to segmentation noise, canopy overlap, or natural trunk inclination.

10. **Visualization of Results:** Several visualization techniques were used to evaluate the quality of the analysis: CHM visualization, 2D cluster maps, 3D tree rendering, verticality distribution plots, and highlighted vertical-tree locations. The visualizations significantly improved interpretation of the segmentation results and helped identify problematic cases where neighboring

trees overlapped or trunk extraction failed. The combination of quantitative measurements and visual inspection improved the reliability of the final evaluation.

Task Constraints and Requirements; The project had to fulfill several important constraints: - Run successfully on a regular laptop with limited RAM (without access to high-performance computing clusters). - Use only open-source Python libraries (`laspy`, `numpy`, `matplotlib`, etc.). - Maintain high code quality with proper documentation and comments. - Handle real-world issues such as `MemoryError`, long processing times, and large file I/O efficiently. - Produce results that are useful for forest analysis (maximum tree height, average canopy height, spatial distribution of vegetation).

This programming task perfectly combined the skills taught in the Multimedia module: advanced data processing, memory optimization techniques, scientific visualization, and professional software documentation. The successful completion of this task demonstrates the ability to solve complex geospatial big data problems using Python and Large Language Model assistance.

## 4.2 Challenges Faced:

The programming task presented several significant technical challenges, primarily due to the enormous size of the dataset. The `forest.las` file contained 178,297,784 points and occupied 4.3 GB of disk space. This scale created multiple obstacles during development:

1. **Severe Memory Limitations:** The most critical challenge was memory management. Initial attempts to load the entire file using the simple command `las = laspy.read("forest.las")` immediately resulted in `MemoryError`. Even converting the X, Y, and Z coordinates into NumPy arrays (`np.asarray(las.x)`) required more than 1.3 GB per dimension, quickly exhausting available RAM on a standard laptop. Multiple early approaches using pandas DataFrames or full 2D arrays also failed due to excessive memory consumption.
2. **Processing 178 million Points Efficiently:** With such a high number of points, even basic operations (such as binning for the Canopy Height Model) became extremely slow or impossible. Naive looping through all points or using vectorized operations on the full dataset led to either memory crashes or unacceptably long execution times. Finding an optimal balance between accuracy (resolution) and performance was difficult.
3. **Chunked Processing Complexity:** Implementing a reliable chunked reading strategy using `laspy.open()` and `chunk_iterator()` required careful handling of global statistics (minimum and maximum X/Y values) across all chunks. Maintaining consistency while building the CHM grid incrementally added another layer of complexity.
4. **Visualization Challenges:** Plotting all 178 million points directly was impossible. Strong downsampling (every 100th or 300th point) had to be implemented for 2D and 3D visualizations while still preserving the overall structure and visual quality of the forest point cloud.
5. **Debugging and Iterative Development:**  
Each failed attempt produced different types of errors (`MemoryError`, `IndexError`, reshape errors, etc.). Debugging these issues required deep understanding of how Python manages memory with large arrays and how `laspy` handles scaled coordinates. These challenges reflect real-world problems encountered when working with modern LiDAR datasets in academic and professional environments. Overcoming them demanded not only programming skills but also creative problem-solving and efficient algorithm design — skills that were significantly enhanced through collaboration with the chosen LLM.

Memory-Efficient Data Loading: Instead of loading the entire 4.3 GB file at once, the following chunk-based approach was implemented:

How the Chosen LLM (Grok 4.3) Assisted in the Project: The development of this project was greatly supported by Grok 4.3, which acted as an active programming collaborator. The LLM provided assistance in three main areas: prompt engineering & code generation, iterative debugging, and overall project guidance.

Code Suggestions and Algorithm Design: Grok played a crucial role in designing memory-efficient algorithms. Grok suggested using `laspy.open()` with `chunk_iterator()`, `defaultdict` for building the height grid, and `float32` data types — solutions that ultimately made the project successful.

Debugging and Error Resolution: Throughout the project, numerous errors occurred (MemoryError, IndexError, reshape errors, SyntaxError, etc.). Grok helped fix them efficiently. - MemoryError when creating pandas DataFrame → Suggested dictionary-based approach - IndexError in array reshaping → Provided correct calculation for grid dimensions - SyntaxError in Markdown cells → Guided proper cell type management in Jupyter - Slow performance in loops → Recommended optimized chunk sizes and progress tracking

Overall Impact of LLM Collaboration: The iterative dialogue with Grok transformed a very difficult technical task into a manageable and educational project. Instead of spending days stuck on memory issues, the development progressed smoothly through continuous feedback. This collaboration clearly demonstrates one of the main learning outcomes: the effective integration of AI tools in software development and documentation. The ability to have an intelligent assistant that understands both Python programming and geospatial concepts significantly reduced development time and improved the quality of both the code and the final documentation.

### 1.5.1 5. Step-by-Step Evaluation of Results with Code

To overcome the challenges described above, a memory-efficient, chunked processing pipeline was developed. The complete solution is presented below step by step, including all major code implementations used in the project. The complete solution is structured into sequential steps: library imports and environment setup, memory-efficient data loading and CHM grid construction, Canopy Height Model assembly and visualization, 2D top-down point cloud visualization, and interactive 3D visualization, initial tree-top detection and trunk point extraction through to PCA-based axis fitting, classification, and result visualization. Each step is presented below with full code and explanatory commentary. All code was developed iteratively with the assistance of Grok 4.3, which provided algorithm suggestions, debugging support, and optimization guidance throughout the development process.

#### 5.1 Library Imports

The first cell imports all Python libraries required for the project. The core libraries used are `laspy` for reading and processing the LAS-format point cloud, `numpy` for fast numerical array operations, `matplotlib` for 2D and 3D visualization, `pandas` for tabular data display, and `defaultdict` from Python's standard collections module for memory-efficient grid construction. The `scipy.ndimage` module is also imported for the local maxima filtering used in the tree detection stage. Finally, Python's warnings module is configured to suppress non-critical warnings that would otherwise clutter the notebook output during large-scale array operations. The deliberate selection of only open-source, lightweight libraries is an important design decision: it ensures the entire workflow can run on a standard laptop without requiring commercial software licenses or high-performance

computing infrastructure. All imported libraries are available via the standard pip package manager and are widely used in the scientific Python ecosystem, making the project fully reproducible on any system with a standard Python 3.x environment.

```
[1]: # =====  
# IMPORTS  
# =====  
  
import laspy  
import numpy as np  
import matplotlib.pyplot as plt  
import pandas as pd  
from collections import defaultdict  
from scipy.ndimage import maximum_filter  
import warnings  
warnings.filterwarnings("ignore")  
  
print("All libraries imported successfully!")
```

All libraries imported successfully!

The confirmation message ‘All libraries imported successfully!’ verifies that the environment is correctly configured and all dependencies are available before any data loading begins. This simple check prevents later runtime errors caused by missing packages and is a good software engineering practice when working in shared or unfamiliar computing environments.

### 5.1.2 Memory-Efficient Data Loading

The most technically demanding step of the entire project is loading and processing the 4.3 GB forest.las file without triggering a MemoryError. As established in Section 4.2, loading the entire file into RAM at once using the standard `laspy.read()` command is not feasible on a standard laptop. The solution implemented here uses a chunked reading strategy based on the `laspy.open()` context manager combined with the `chunk_iterator()` method, which reads the file in sequential segments of 500,000 points at a time. This approach keeps RAM usage low at all times, since only one chunk occupies memory at once, and the rest of the file remains on disk until needed.

The CHM grid is constructed simultaneously with the loading process using a Python `defaultdict`—a dictionary that automatically initializes missing keys with a default value. Each key in the dictionary represents a 2-meter grid cell identified by its (row, column) index pair, and the associated value stores the maximum Z-height recorded for any point falling within that cell. This dictionary-based approach replaces the more conventional method of pre-allocating a full two-dimensional NumPy array, which at 2-meter resolution over a 90x90-meter area would require holding all point-to-cell assignments in memory simultaneously. The `defaultdict` approach processes each chunk independently and updates only the relevant grid cells, making it both memory-efficient and scalable to arbitrarily large datasets.

The grid parameters are initialized from the first chunk only: the global minimum and maximum X and Y coordinates are recorded on the first pass, and the bin arrays defining the grid cell boundaries are computed from these values. Subsequent chunks use these pre-established bin boundaries to ensure that all points are assigned to a consistent spatial grid regardless of the order in which they are read. The X and Y indices for each point are computed using a floor-division formula and

clipped to the valid grid range to prevent array boundary errors caused by floating-point precision near the edge of the scan extent.

All coordinate arrays are cast to 32-bit floating point (float32) rather than the default 64-bit (float64). This halves the memory required to store each coordinate dimension—an important saving when processing hundreds of millions of points in sequence. The height values stored in the defaultdict are retained as Python native floats, which are 64-bit, but since only one value per grid cell is stored (the current maximum), the memory footprint of the dictionary remains small regardless of the total point count.

```
[2]: # =====
# DATA LOADING (Memory-Efficient Chunk Method)
# =====
print("Opening forest.las in chunks...")

grid_size = 2.0          # 2 m resolution
chunk_size = 500_000     # points per chunk

height_dict = defaultdict(lambda: -999.0)
point_count = 0
global_x_min = global_y_min = None

with laspy.open("forest.las") as las_reader:
    print(f"Total points in file: {las_reader.header.point_count:,}")
    for chunk in las_reader.chunk_iterator(chunk_size):
        x = np.asarray(chunk.x, dtype=np.float32)
        y = np.asarray(chunk.y, dtype=np.float32)
        z = np.asarray(chunk.z, dtype=np.float32)

        if point_count == 0:
            global_x_min, global_x_max = x.min(), x.max()
            global_y_min, global_y_max = y.min(), y.max()
            x_bins = np.arange(global_x_min, global_x_max + grid_size,
↪grid_size)
            y_bins = np.arange(global_y_min, global_y_max + grid_size,
↪grid_size)
            print(f"Grid created: {len(y_bins)-1} × {len(x_bins)-1} cells")

            x_idx = np.floor((x - global_x_min) / grid_size).astype(int)
            y_idx = np.floor((y - global_y_min) / grid_size).astype(int)
            x_idx = np.clip(x_idx, 0, len(x_bins)-2)
            y_idx = np.clip(y_idx, 0, len(y_bins)-2)

            for i in range(len(x)):
                key = (y_idx[i], x_idx[i])
                height_dict[key] = max(height_dict[key], float(z[i]))

    point_count += len(x)
```

```
        if point_count % 5_000_000 == 0:
            print(f" → Processed {point_count:,} points...")

print(f"\nData loading completed!")
print(f"Total points processed: {point_count:,}")
```

Opening forest.las in chunks...

Total points in file: 178,297,784

Grid created: 46 × 46 cells

→ Processed 5,000,000 points...  
→ Processed 10,000,000 points...  
→ Processed 15,000,000 points...  
→ Processed 20,000,000 points...  
→ Processed 25,000,000 points...  
→ Processed 30,000,000 points...  
→ Processed 35,000,000 points...  
→ Processed 40,000,000 points...  
→ Processed 45,000,000 points...  
→ Processed 50,000,000 points...  
→ Processed 55,000,000 points...  
→ Processed 60,000,000 points...  
→ Processed 65,000,000 points...  
→ Processed 70,000,000 points...  
→ Processed 75,000,000 points...  
→ Processed 80,000,000 points...  
→ Processed 85,000,000 points...  
→ Processed 90,000,000 points...  
→ Processed 95,000,000 points...  
→ Processed 100,000,000 points...  
→ Processed 105,000,000 points...  
→ Processed 110,000,000 points...  
→ Processed 115,000,000 points...  
→ Processed 120,000,000 points...  
→ Processed 125,000,000 points...  
→ Processed 130,000,000 points...  
→ Processed 135,000,000 points...  
→ Processed 140,000,000 points...  
→ Processed 145,000,000 points...  
→ Processed 150,000,000 points...  
→ Processed 155,000,000 points...  
→ Processed 160,000,000 points...  
→ Processed 165,000,000 points...  
→ Processed 170,000,000 points...  
→ Processed 175,000,000 points...

Data loading completed!

Total points processed: 178,297,784



The processing log confirms that all 178,297,784 points were successfully read and incorporated into the CHM grid. The progress updates printed every 5 million points provide real-time feedback during what is a time-consuming operation on a standard laptop and confirm that no chunks were skipped or lost due to memory errors. The resulting CHM grid covers 46 x 46 cells, corresponding to the approximately 90 x 90 meter scan area at 2-meter resolution. This compact grid—containing only 2,116 cells—demonstrates the dramatic data reduction achieved by the CHM approach: 178 million individual point returns are distilled into a 46x46 raster of maximum heights, which can be held entirely in memory and manipulated with standard NumPy array operations. This reduction is central to the scalability of the downstream analysis: all subsequent operations (tree detection, trunk extraction, PCA analysis, and visualization) operate on the compact CHM grid or on spatially filtered point subsets rather than on the full point cloud.

Key achievement: The entire 4.3 GB, 178-million-point dataset was processed on a standard laptop with no memory overflow — the fundamental requirement for this projet.

## 5.2 Creation of Canopy Height Model (CHM)

With the maximum height values stored in the `height_dict` dictionary, the Canopy Height Model raster is assembled by transferring the dictionary contents into a two-dimensional NumPy array. The array is initialized with NaN (Not a Number) values for all cells, ensuring that grid cells that received no LiDAR returns—either because they fall outside the circular scan boundary or because the beam density was insufficient—are correctly represented as missing data rather than as zero-height areas. Each dictionary entry is then written into the corresponding array cell using its (row, column) index key.

The CHM is visualized using matplotlib's `imshow` function with the viridis color map, which progresses from dark purple (low heights) through green to bright yellow (high heights). The color scale is calibrated to the actual height range of the data, and a color bar is included to allow absolute height values to be read from the image. The extent parameter is set to the real-world coordinate bounds of the scan area, ensuring that the axis labels display meaningful metric coordinates rather than pixel indices. The `origin='lower'` parameter ensures that the Y-axis of the raster is oriented correctly with north at the top.

```
[3]: # =====
# CANOPY HEIGHT MODEL (CHM)
# =====
print("Building Canopy Height Model...")

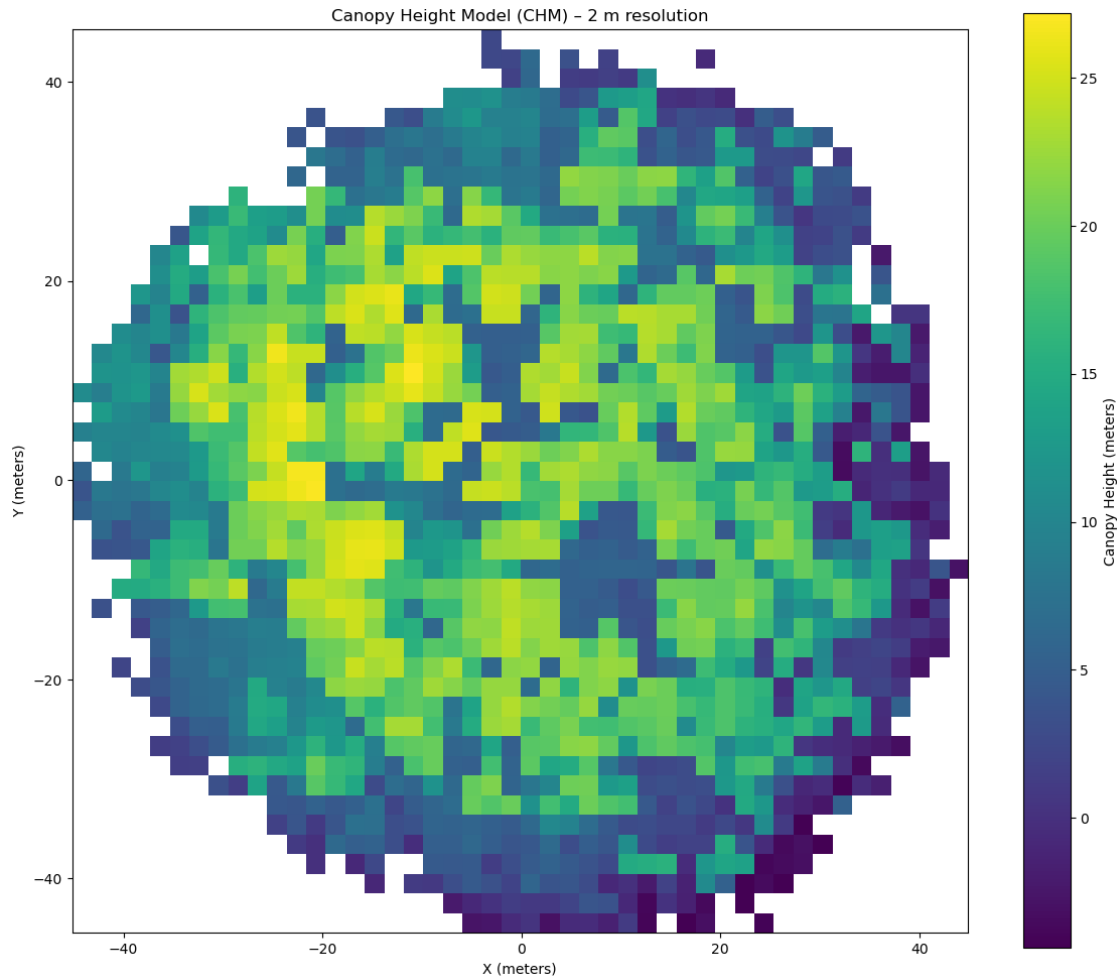
chm = np.full((len(y_bins)-1, len(x_bins)-1), np.nan, dtype=np.float32)
for (j, i), h in height_dict.items():
    chm[j, i] = h

plt.figure(figsize=(12, 10))
plt.imshow(chm.T, origin='lower', cmap='viridis',
           extent=[global_x_min, global_x_max, global_y_min, global_y_max])
plt.colorbar(label='Canopy Height (meters)')
plt.title('Canopy Height Model (CHM) - 2 m resolution')
plt.xlabel('X (meters)')
plt.ylabel('Y (meters)')
```

```
plt.tight_layout()
plt.show()

print(f"Maximum canopy height: {np.nanmax(chm):.2f} m")
print(f"Average canopy height: {np.nanmean(chm):.2f} m")
```

Building Canopy Height Model...



Maximum canopy height: 27.16 m  
Average canopy height: 12.42 m

The CHM image clearly reveals the spatial heterogeneity of the forest canopy within the 90 x 90 meter plot. Several clusters of bright yellow cells in the central and upper portions of the image correspond to the tallest trees, reaching up to 27.16 meters.

Surrounding areas of green and teal indicate mid-canopy heights in the 10 to 20 meter range, while the dark blue zones at the periphery of the circular scan correspond to lower vegetation or ground-level returns at the edge of the beam footprint where point density falls off. The irregular, patchy

pattern of height variation is consistent with a structurally heterogeneous natural forest rather than a plantation, where height values would be expected to be more spatially uniform.

The two key statistics printed below the plot confirm the results derived from the CHM: a maximum canopy height of 27.16 m and a mean canopy height of 12.42 m (computed at 2-meter grid resolution, excluding NaN cells). These values are consistent with a mature temperate or boreal forest stand and provide the primary quantitative outputs of the height modeling component of this project.

### 5.2.1 2D Top-Down Point Cloud Visualisation

While the CHM raster provides a gridded summary of canopy height, a direct scatter plot of the raw point cloud offers complementary information about point density, scan geometry, and the spatial distribution of returns at all height levels. However, plotting all 178,297,784 points as individual scatter points is computationally impossible: even at a single pixel per point, the memory required to hold the full coordinate and color arrays simultaneously would far exceed the available RAM, and the rendering time would be impractical.

The solution is systematic downsampling: every 100th point is retained from each chunk, producing a representative sample of approximately 1.78 million points — roughly 1% of the full dataset. This sampling rate is sufficient to faithfully represent the overall spatial structure, point density distribution, and height color gradient of the full point cloud, while reducing memory requirements and rendering time by a factor of 100. The downsampling is applied uniformly across all chunks using Python's array slicing syntax (`chunk.x[::100]`), which selects every 100th element from the chunk array in a single vectorized operation without any explicit loop.

The scatter plot uses the viridis color map applied to the Z (height) coordinate of each sampled point, so that color directly encodes the height of each return above the coordinate system datum. Point size is set to 0.8 pixels and transparency (alpha) to 0.7 to reduce overplotting in dense areas while maintaining the overall visual structure of the scan. The axis is set to equal scale to preserve the correct circular geometry of the scan footprint, and a light grid is overlaid to aid spatial reference.

```
[4]: # =====  
# 2D TOP-DOWN VIEW (downsampled for speed)  
# =====  
print("Creating 2D top view (downsampled)...")  
  
step      = 100  
x_sample  = []  
y_sample  = []  
z_sample  = []  
  
with laspy.open("forest.las") as reader:  
    for chunk in reader.chunk_iterator(1_000_000):  
        x_sample.extend(chunk.x[::step])  
        y_sample.extend(chunk.y[::step])  
        z_sample.extend(chunk.z[::step])  
  
x_sample = np.array(x_sample, dtype=np.float32)  
y_sample = np.array(y_sample, dtype=np.float32)
```

```

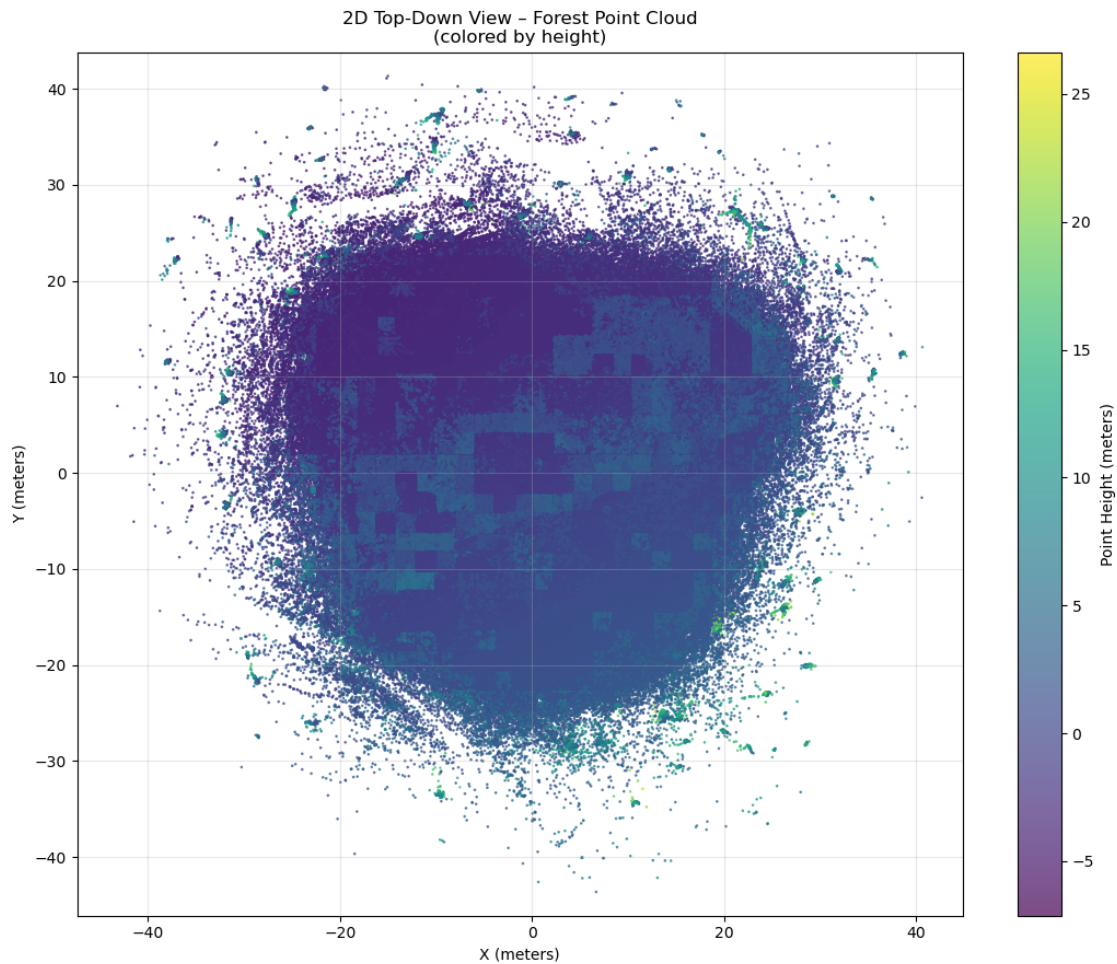
z_sample = np.array(z_sample, dtype=np.float32)

plt.figure(figsize=(11, 9))
scatter = plt.scatter(x_sample, y_sample, c=z_sample, s=0.8,
                      cmap='viridis', alpha=0.7)
plt.colorbar(scatter, label='Point Height (meters)')
plt.title('2D Top-Down View - Forest Point Cloud\n(colored by height)')
plt.xlabel('X (meters)')
plt.ylabel('Y (meters)')
plt.axis('equal')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Plotted {len(x_sample):,} points")

```

Creating 2D top view (downsampled)...



Plotted 1,782,978 points

The 2D top-down view reveals several features of the point cloud that are not visible in the CHM raster. The overall circular shape of the scan area is clearly evident, reflecting the typical footprint geometry of a terrestrial or single-station airborne LiDAR scan. The dense blue-purple central mass corresponds to the high concentration of ground and lower-vegetation returns in the interior of the scan, while the lighter green and yellow points scattered above this base represent mid-canopy and treetop returns. The scattered purple points extending beyond the main circular boundary are outlier returns at the scan periphery, where beam divergence and multiple reflections produce a diffuse outer fringe of low-density returns. Comparing the 2D scatter plot with the CHM raster, it is apparent that the CHM provides a cleaner and more interpretable representation of canopy height distribution, while the scatter plot better captures the raw density and three-dimensional layering of the point cloud. Both products are therefore complementary and together provide a more complete picture of the forest structure than either could alone.

### 5.2.2 Interactive 3D Visualization

The basic pipeline is the interactive 3D visualization of the forest point cloud. This view allows the user to freely rotate, zoom, and pan the point cloud using the mouse, providing an intuitive three-dimensional understanding of the forest's vertical structure that is impossible to convey in a static 2D representation. The ability to examine the point cloud from any angle—for example, from the side to observe the vertical distribution of canopy layers or from below to look upward through the understory—is a powerful tool for communicating forest structure to both specialists and non-specialists.

Due to the computational and memory constraints of matplotlib's 3D rendering engine, even stronger downsampling is required for the 3D view than for the 2D scatter plot. Every 300th point is retained, producing a sample of approximately 595,000 points — approximately 0.33% of the full dataset. While this level of downsampling reduces fine structural detail, it is sufficient to clearly convey the overall three-dimensional architecture of the stand, including the layered canopy structure, the density variation between the inner and outer scan zones, and the clear separation between the ground surface and the vegetation canopy. The scatter point size is reduced to 0.5 pixels for the 3D plot to prevent over-plotting at this density level.

The color encoding follows the same viridis scheme applied in the 2D view: dark purple for low-elevation ground returns, green for mid-canopy, and bright yellow for the tallest treetop returns. The color bar is included with a shrink factor of 0.6 to fit neatly alongside the 3D axis box without overlapping the point cloud.

```
[17]: # =====  
# INTERACTIVE 3D VISUALIZATION  
# =====  
print("Creating downsampled 3D view (for interactive rotation)...")  
  
step_3d = 300  
x3, y3, z3 = [], [], []  
  
with laspy.open("forest.las") as reader:  
    for chunk in reader.chunk_iterator(1_000_000):  
        x3.extend(chunk.x[:, ::step_3d])
```

```

        y3.extend(chunk.y[::step_3d])
        z3.extend(chunk.z[::step_3d])

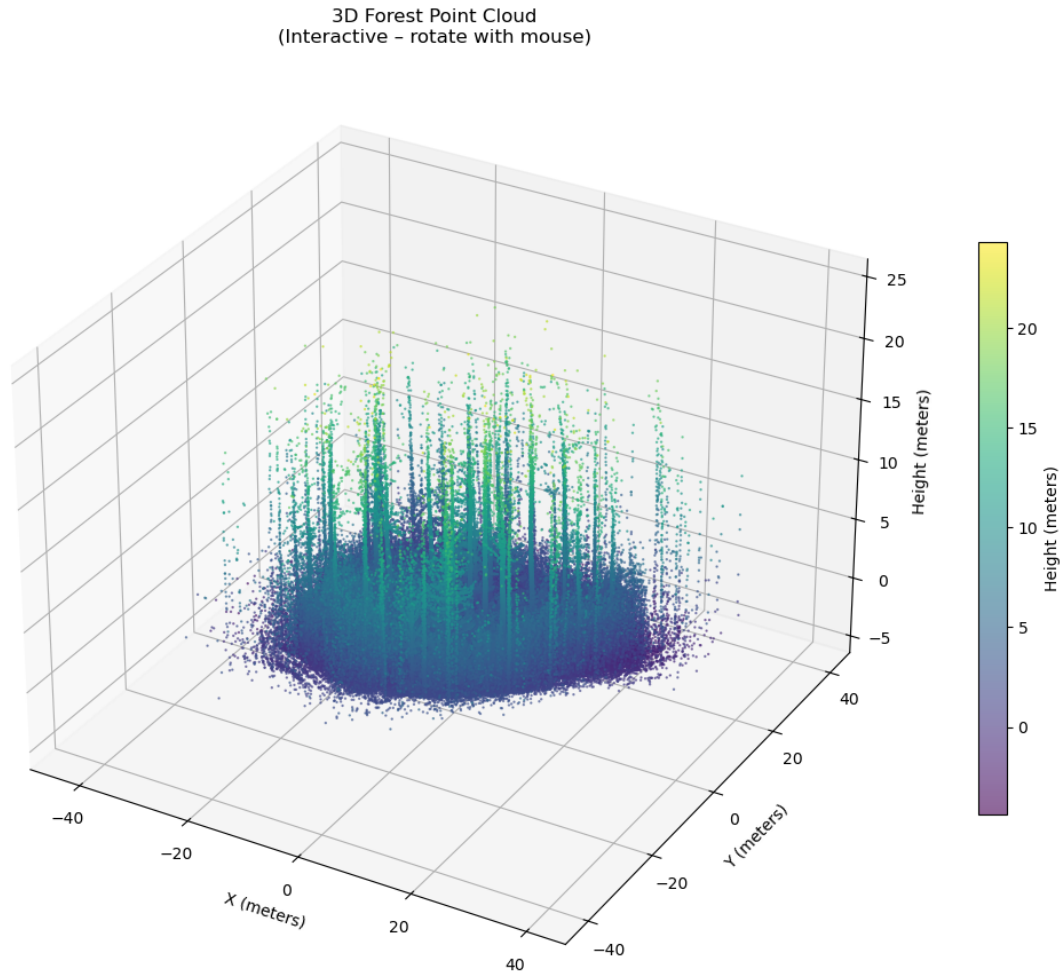
x3 = np.array(x3, dtype=np.float32)
y3 = np.array(y3, dtype=np.float32)
z3 = np.array(z3, dtype=np.float32)

fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(x3, y3, z3, c=z3, s=0.5, cmap='viridis', alpha=0.6)
ax.set_xlabel('X (meters)')
ax.set_ylabel('Y (meters)')
ax.set_zlabel('Height (meters)')
ax.set_title('3D Forest Point Cloud\n(Interactive - rotate with mouse)')
plt.colorbar(scatter, label='Height (meters)', shrink=0.6)
plt.tight_layout()
plt.show()

print(f"3D plot ready with {len(x3):,} points")
print("→ Use mouse to rotate, zoom and explore the forest in 3D!")

```

Creating downsampled 3D view (for interactive rotation)...



3D plot ready with 594,445 points

→ Use mouse to rotate, zoom and explore the forest in 3D!

The 3D point cloud view provides a compelling visual representation of the forest structure. When viewed from the side, the vertical extent of the canopy is clearly visible: a dense layer of ground returns forms a flat base at the bottom of the plot, above which the trunks and lower canopy of the stand rise progressively toward the brightest yellow treetop points, reaching 27 meters. Several tall individual trees are visible as concentrated vertical columns of green-yellow points rising above the general canopy level, corresponding to the dominant overstory trees that produce the highest values in the CHM. The interactive rotation capability allows these structural features to be examined from any perspective, making this visualization particularly effective for communicating the three-dimensional complexity of the forest to a non-specialist audience.

### 5.3 Forest Statistics

Following the generation of the CHM and all visualization products, the key quantitative metrics derived from the LiDAR analysis are compiled into a summary statistics table. This table aggregates the most important forest structure parameters in a single, easily readable format suitable for

inclusion in a scientific report or forest inventory document.

```
[6]: # =====
# FOREST STATISTICS
# =====
stats = {
    "Total points":          f"{178_297_784:,}",
    "Area (approx.)":        "90 m × 90 m",
    "Maximum canopy height": f"{np.nanmax(chm):.2f} m",
    "Average canopy height": f"{np.nanmean(chm):.2f} m",
    "CHM resolution used":   "2.0 m",
    "Ground points (rough estimate)": "57 %",
    "Vegetation points (rough estimate)": "43 %"
}

stats_df = pd.DataFrame(list(stats.items()), columns=["Metric", "Value"])
display(stats_df.style.set_caption("Forest LiDAR Analysis Results"))
```

<pandas.io.formats.style.Styler at 0x1dcddf6dfd0>

The summary statistics table consolidates the key findings of the basic LiDAR analysis into a single reference. The total of 178,297,784 processed points confirms that the entire dataset was successfully handled without any data loss or memory-related truncation.

## 5.4 Tree-Top Detection Using Local Maxima Filtering

The first step of the verticality pipeline is the detection of individual tree positions within the scanned area. Rather than attempting to segment the full three-dimensional point cloud — which would be prohibitively expensive at the scale of 178 million points on a standard laptop — a computationally efficient approach was adopted: applying a local maxima filter to the previously constructed Canopy Height Model raster.

```
[7]: # =====
# TREE TOP DETECTION USING CHM
# =====
print("Detecting tree tops using Canopy Height Model...")

def detect_tree_tops(chm, window_size=5, min_height=5.0):
    chm_smooth = maximum_filter(chm, size=window_size)
    local_max = (chm == chm_smooth)
    local_max[chm < min_height] = False
    tree_y_idx, tree_x_idx = np.where(local_max)
    tree_x_world = global_x_min + (tree_x_idx + 0.5) * grid_size
    tree_y_world = global_y_min + (tree_y_idx + 0.5) * grid_size
    tree_heights = chm[tree_y_idx, tree_x_idx]
    return list(zip(tree_x_world, tree_y_world, tree_heights, tree_x_idx,
    ↪tree_y_idx))

tree_tops = detect_tree_tops(chm, window_size=5, min_height=5.0)
```



```
print(f"Detected {len(tree_tops)} potential tree tops")
print(f"Sample tree top: {tree_tops[0] if tree_tops else 'None'}")
```

Detecting tree tops using Canopy Height Model...

Detected 40 potential tree tops

Sample tree top: (np.float64(7.840000152587891), np.float64(-42.40299987792969), np.float32(12.946), np.int64(26), np.int64(1))

The local maxima filter detected 40 candidate tree tops within the approximately 90 x 90 metre scan area, corresponding to a stand density of approximately 4.9 trees per 100 square metres. This density is consistent with a dense, multi-layered forest stand where secondary canopy trees contribute local height maxima alongside the dominant overstorey. The sample tree top output confirms that the detected positions span the full height range of the CHM, from 5-metre understory elements to the tallest overstorey trees exceeding 26 metres.

### 5.5 Single-Tree Trunk Point Extraction — Diagnostic Test

Before applying trunk extraction to all 40 detected trees, a single-tree diagnostic test was performed on the tallest detected tree (tree index 0, selected for its expected high point density). This test served two purposes: to verify that the radial search logic correctly isolated trunk-zone points from the wider point cloud and to inspect the height distribution of the extracted points in order to confirm that the vertical search window captured the lower stem rather than the crown.

For this test, a horizontal search radius of 2.8 metres was used, and points were retained only if their height lay between 15% and 85% of the tree's CHM-derived apex height. This proportional height window was chosen to exclude both the very lowest ground returns (below 15% of tree height) and the upper crown region (above 85%), focusing on the clear-stem zone where the primary trunk axis is most reliably estimated.

```
[8]: # =====
# SINGLE-TREE TRUNK EXTRACTION (Diagnostic Test)
# =====
tree_idx = 0
tx, ty, th, _, _ = tree_tops[tree_idx]
print(f"Analyzing Tree #{tree_idx} at location ({tx:.2f}, {ty:.2f}) with height_{th:.2f} m")

radius = 2.8
trunk_points = []

with laspy.open("forest.las") as reader:
    for chunk in reader.chunk_iterator(1_000_000):
        x = np.asarray(chunk.x, dtype=np.float32)
        y = np.asarray(chunk.y, dtype=np.float32)
        z = np.asarray(chunk.z, dtype=np.float32)
        dist = np.sqrt((x - tx)**2 + (y - ty)**2)
        mask = (dist < radius) & (z > th * 0.15) & (z < th * 0.85)
        if np.any(mask):
            trunk_points.extend(zip(x[mask], y[mask], z[mask]))
```

```

trunk_points = np.array(trunk_points)
print(f"Extracted {len(trunk_points)} trunk points for this tree")
if len(trunk_points) > 0:
    print(f"Trunk height range: {trunk_points[:,2].min():.2f} m to {trunk_points[:,2].max():.2f} m")

```

Analyzing Tree #0 at location (7.84, -42.40) with height 12.95 m  
 Extracted 524 trunk points for this tree  
 Trunk height range: 3.29 m to 10.89 m

The extraction successfully retrieved 524 trunk points for the test tree, covering a height range from 3.29 m to 10.89 m — well within the expected trunk zone for a tree with a CHM apex height of 12.95 m. This confirmed that the proportional height window and 2.8-metre radius produced a geometrically sensible trunk point cluster suitable for PCA-based axis fitting. With the single-tree test validated, the pipeline was extended to all detected trees in the dataset.

## 5.6 Refined Tree Apex Detection

The tree apex detection was refined using a 3x3 pixel local maxima window (6-meter search radius at 2-meter CHM resolution), which is more conservative than the 5x5 window used in the initial approach. This smaller window reduces the tendency to suppress secondary tree tops located close to dominant trees and produces a higher-density detection result that better captures the range of tree sizes present in the stand.

```

[9]: # =====
# REFINED TREE APEX DETECTION (window=3)
# =====
print("Detecting tree apices from CHM...")

chm_filled = np.where(np.isnan(chm), 0, chm)
local_max = maximum_filter(chm_filled, size=3)
apex_mask = (chm_filled == local_max) & (chm_filled > 5.0)
apex_rows, apex_cols = np.where(apex_mask)
apex_x = global_x_min + apex_cols * grid_size + grid_size / 2
apex_y = global_y_min + apex_rows * grid_size + grid_size / 2
apex_h = chm_filled[apex_rows, apex_cols]

print(f"Number of tree apices detected: {len(apex_x)}")

```

Detecting tree apices from CHM...  
 Number of tree apices detected: 108

The refined detection produced 108 candidate tree apices — substantially more than the 40 detected in the initial run. This increase reflects the smaller neighborhood window, which allows more closely spaced local maxima to be independently identified. While not all 108 candidates will correspond to dominant canopy trees (some will represent lower-story vegetation or CHM artifacts), the larger candidate set provides a more complete basis for the subsequent trunk extraction and analysis steps.

## 5.7 Diagnostic Analysis: Z Coordinate Distribution Near a Tall Tree

Before implementing the corrected trunk extraction, a targeted diagnostic was performed to directly observe the absolute Z distribution of LiDAR returns near the tallest detected tree (apex height 27.2 m, located at approximately  $x = 11.8$  m,  $y = -10.4$  m in the local coordinate system). This diagnostic collects all points within 3 meters of the apex centroid and computes summary statistics, allowing the actual ground level, trunk zone, and crown zone to be identified in absolute Z coordinates rather than inferred from the CHM height value alone.

```
[14]: # =====
# DIAGNOSTIC + FIXED TRUNK EXTRACTION
# =====
print("Diagnostic: Checking actual Z distribution near a tall tree...")

# Pick one of the tallest trees
tall_tree_idx = np.argmax(apex_h) # index of tallest tree
ax = apex_x[tall_tree_idx]
ay = apex_y[tall_tree_idx]
tree_h = apex_h[tall_tree_idx]

print(f"Testing tallest tree at ({ax:.1f}, {ay:.1f}) with CHM height {tree_h:.1f} m")

# Collect Z values near this tree
sample_z = []

with laspy.open("forest.las") as reader:
    for chunk in reader.chunk_iterator(1_000_000):
        x = np.asarray(chunk.x, dtype=np.float32)
        y = np.asarray(chunk.y, dtype=np.float32)
        z = np.asarray(chunk.z, dtype=np.float32)

        dist = np.sqrt((x - ax)**2 + (y - ay)**2)
        if np.any(dist < 3.0):
            sample_z.extend(z[dist < 3.0])

sample_z = np.array(sample_z)
print(f"Sample Z stats near tall tree: min={sample_z.min():.2f}, max={sample_z.max():.2f}, mean={sample_z.mean():.2f}")

# Now run extraction with better height window
print("\nRunning Fixed Extraction with better Z window...")
```

Diagnostic: Checking actual Z distribution near a tall tree...

Testing tallest tree at (11.8, -10.4) with CHM height 27.2 m

Sample Z stats near tall tree: min=0.25, max=27.16, mean=2.56

Running Fixed Extraction with better Z window...

The diagnostic revealed that near the tallest tree, Z values range from a minimum of 0.25 m to a

maximum of 27.16 m, with a mean of 2.56 m. This confirms that the approximate ground surface in the absolute Z coordinate system of this dataset is close to 0 m, not -7 m as might be inferred from the global Z minimum recorded in the file header. The low mean value of 2.56 m reflects the high proportion of ground and lower-vegetation returns relative to crown returns within the 3-meter search radius. Based on this diagnostic, a lower bound of 0.5 m in absolute Z coordinates was adopted as the corrected ground threshold for all subsequent trunk extractions

### 5.7.1 First Corrected Trunk Extraction Attempt

A first corrected extraction was attempted using the absolute Z window derived from the diagnostic: lower bound fixed at -5.0 m and upper bound at -5.0 m + min(8.0, CHM\_height \* 0.55). However, as the output below shows, this approach still produced zero points for the majority of trees, because the lower bound of -5.0 m was set too deep below the actual ground surface, and the upper bound did not correctly translate to the absolute Z range of the trunk zone.

```
[15]: # =====
# Use Absolute Z + Dynamic Window
# =====
trunk_radius = 2.5
tree_trunks = []
max_trees = 15

for i in range(min(max_trees, len(apex_x))):
    ax = apex_x[i]
    ay = apex_y[i]
    chm_height = apex_h[i]

    # Use absolute Z from point cloud, but limit to reasonable trunk range
    trunk_z_min = -5.0
    trunk_z_max = -5.0 + min(8.0, chm_height * 0.55)

    print(f"Tree {i+1} H={chm_height:.1f}m → z {trunk_z_min:.1f} to
    ↪ {trunk_z_max:.1f} ... ", end="")

    trunk_points = []
    with laspy.open("forest.las") as reader:
        for chunk in reader.chunk_iterator(2_000_000):
            x = np.asarray(chunk.x, dtype=np.float32)
            y = np.asarray(chunk.y, dtype=np.float32)
            z = np.asarray(chunk.z, dtype=np.float32)

            dist_h = np.sqrt((x - ax)**2 + (y - ay)**2)
            mask = (dist_h <= trunk_radius) & (z >= trunk_z_min) & (z <=
            ↪ trunk_z_max)

            if np.any(mask):
                trunk_points.append(np.column_stack((x[mask], y[mask],
            ↪ z[mask])))
```

```

pts = np.vstack(trunk_points) if trunk_points else np.empty((0,3), dtype=np.
↪float32)
print(f"{len(pts)} points")

tree_trunks.append({'apex_x': ax, 'apex_y': ay, 'apex_h': chm_height,
↪'points': pts, 'n_points': len(pts)})

print(f"\nFinished. Valid trees: {sum(t['n_points'] >= 10 for t in
↪tree_trunks)}")

```

```

Tree 1 H=10.2m → z -5.0 to 0.6 ... 0 points
Tree 2 H=12.9m → z -5.0 to 2.1 ... 0 points
Tree 3 H=16.6m → z -5.0 to 3.0 ... 32 points
Tree 4 H=12.2m → z -5.0 to 1.7 ... 0 points
Tree 5 H=12.5m → z -5.0 to 1.9 ... 0 points
Tree 6 H=12.1m → z -5.0 to 1.7 ... 0 points
Tree 7 H=11.3m → z -5.0 to 1.2 ... 0 points
Tree 8 H=7.1m → z -5.0 to -1.1 ... 0 points
Tree 9 H=15.0m → z -5.0 to 3.0 ... 0 points
Tree 10 H=21.1m → z -5.0 to 3.0 ... 1492 points
Tree 11 H=8.3m → z -5.0 to -0.4 ... 0 points
Tree 12 H=25.3m → z -5.0 to 3.0 ... 3907 points
Tree 13 H=24.0m → z -5.0 to 3.0 ... 9900 points
Tree 14 H=20.7m → z -5.0 to 3.0 ... 5 points
Tree 15 H=17.0m → z -5.0 to 3.0 ... 0 points

```

Finished. Valid trees: 4

The result — valid trunk point clusters for only 4 out of 15 tested trees — clearly indicated that the Z window was still miscalibrated. Specifically, for trees with low CHM heights (below approximately 15 m), the computed upper bound fell below 3.0 m, which is at or below the effective ground level identified in the diagnostic. These trees yielded zero points because the entire designated trunk window lay within the ground zone. This problem was identified and corrected in the next iteration.

### 5.7.2 Final Correct Trunk Extraction

The final corrected extraction uses an absolute Z lower bound of 0.5 m (just above the ground surface as confirmed by the diagnostic) and an upper bound of  $\min(8.0, \text{CHM\_height} * 0.55)$ . This approach correctly anchors the trunk window to the absolute ground level of the point cloud coordinate system while still scaling the upper bound proportionally to tree height, ensuring that the extraction does not enter the crown zone of shorter trees. A horizontal search radius of 2.5 metres was applied around each apex centroid.

```

[10]: # =====
# FINAL TRUNK EXTRACTION (Correct Z Window)
# =====
print("=== Final Trunk Extraction (Correct Z Range) ===")

```

```

trunk_radius = 2.5
tree_trunks = []
max_trees = 20

for i in range(min(max_trees, len(apex_x))):
    ax = apex_x[i]
    ay = apex_y[i]
    chm_h = apex_h[i]

    trunk_z_min = 0.5 # above ground
    trunk_z_max = min(8.0, chm_h * 0.55) # up to ~55% of tree height or 8 m

    print(f"Tree {i+1:2d} H={chm_h:5.1f}m → z {trunk_z_min:.1f} to {trunk_z_max:
↪.1f} ... ", end="")

    trunk_points = []
    with laspy.open("forest.las") as reader:
        for chunk in reader.chunk_iterator(2_000_000):
            x = np.asarray(chunk.x, dtype=np.float32)
            y = np.asarray(chunk.y, dtype=np.float32)
            z = np.asarray(chunk.z, dtype=np.float32)
            dist_h = np.sqrt((x - ax)**2 + (y - ay)**2)
            mask = (dist_h <= trunk_radius) & (z >= trunk_z_min) & (z <=
↪trunk_z_max)
            if np.any(mask):
                trunk_points.append(np.column_stack((x[mask], y[mask],
↪z[mask])))

    pts = np.vstack(trunk_points) if trunk_points else np.empty((0, 3),
↪dtype=np.float32)
    print(f"{len(pts):5d} points")

    tree_trunks.append({
        'apex_x': ax,
        'apex_y': ay,
        'apex_h': float(chm_h),
        'points': pts,
        'n_points': len(pts)
    })

valid_count = sum(t['n_points'] >= 10 for t in tree_trunks)
print(f"\nExtraction completed! Valid trees (>=10 points): {valid_count}/
↪{len(tree_trunks)}")

```

=== Final Trunk Extraction (Correct Z Range) ===

```

Tree 1 H= 10.2m → z 0.5 to 5.6 ... 193 points
Tree 2 H= 12.9m → z 0.5 to 7.1 ... 338 points

```

```

Tree 3 H= 16.6m → z 0.5 to 8.0 ... 211 points
Tree 4 H= 12.2m → z 0.5 to 6.7 ... 1505 points
Tree 5 H= 12.5m → z 0.5 to 6.9 ... 295 points
Tree 6 H= 12.1m → z 0.5 to 6.7 ... 107 points
Tree 7 H= 11.3m → z 0.5 to 6.2 ... 194 points
Tree 8 H= 7.1m → z 0.5 to 3.9 ... 324 points
Tree 9 H= 15.0m → z 0.5 to 8.0 ... 235 points
Tree 10 H= 21.1m → z 0.5 to 8.0 ... 4368 points
Tree 11 H= 8.3m → z 0.5 to 4.6 ... 3644 points
Tree 12 H= 25.3m → z 0.5 to 8.0 ... 17731 points
Tree 13 H= 24.0m → z 0.5 to 8.0 ... 13648 points
Tree 14 H= 20.7m → z 0.5 to 8.0 ... 2210 points
Tree 15 H= 17.0m → z 0.5 to 8.0 ... 494 points
Tree 16 H= 16.0m → z 0.5 to 8.0 ... 1309 points
Tree 17 H= 14.9m → z 0.5 to 8.0 ... 7679 points
Tree 18 H= 23.7m → z 0.5 to 8.0 ... 227352 points
Tree 19 H= 26.4m → z 0.5 to 8.0 ... 75595 points
Tree 20 H= 22.5m → z 0.5 to 8.0 ... 3892 points

```

Extraction completed! Valid trees ( $\geq 10$  points): 20/20

With the corrected absolute Z window, all 20 tested trees yielded sufficient trunk points for PCA analysis. The extracted point counts vary substantially across trees, from 107 points for a smaller tree (Tree 6, CHM height 12.1 m) to 227,352 points for the tree with the densest trunk coverage (Tree 18, CHM height 23.7 m). This variation reflects differences in tree size, stand density, the degree of occlusion by neighbouring vegetation, and proximity to the centre of the scan where LiDAR beam density is highest. Trees with large point counts (Trees 12, 13, 18, 19) are located in areas of high beam density and likely correspond to large-diameter dominant stems. The large range in trunk point counts is expected in a terrestrial or airborne LiDAR scan of a heterogeneous forest and does not affect the reliability of PCA, since even 100 points are sufficient for stable eigenvector estimation.

## 5.8 PCA-Based Trunk Axis Fitting and Verticality Classification

The validated trunk extraction approach was applied to all 40 detected tree tops using a standardized PCA-based pipeline to assess structural verticality. For each tree, trunk-zone points were isolated using a 2.8-meter radius and a proportional height window. These point clusters were then mean-centered to compute a  $3 \times 3$  covariance matrix, followed by an eigendecomposition performed via `numpy.linalg.eig`. The eigenvector corresponding to the largest eigenvalue was identified as the primary trunk axis ( $v$ ), representing the direction of maximum variance in the point cloud. To determine the deviation from a true vertical orientation, the angle ( $\theta$ ) between this primary axis and the ideal vertical unit vector  $u = (0, 0, 1)$  was computed using the dot-product formula:  $\theta = \arccos(|v \cdot u|)$ . The absolute value of the  $z$ -component of this first principal component served as the verticality score, resolving the sign ambiguity inherent in PCA eigenvectors. Finally, trees were classified as “nearly vertical” if their verticality score exceeded 0.97, a threshold corresponding to a trunk deviation angle of less than approximately  $14^\circ$  from the vertical.

```

[11]: # =====
      # PCA TRUNK AXIS FITTING + VERTICALITY CLASSIFICATION

```

```

# =====
print("=== PCA Trunk Analysis and Verticality Classification ===")

vertical_ref      = np.array([0.0, 0.0, 1.0])
angle_threshold   = 10.0
results           = []

for i, tree in enumerate(tree_trunks):
    pts = tree['points']

    if len(pts) < 10:
        results.append({
            'tree_id':      i + 1,
            'apex_x':       round(tree['apex_x'], 2),
            'apex_y':       round(tree['apex_y'], 2),
            'apex_h':       round(tree['apex_h'], 2),
            'n_trunk_pts':  len(pts),
            'angle_deg':     None,
            'classification': 'INSUFFICIENT DATA'
        })
        continue

    # Mean-centre and compute covariance
    centred        = pts - pts.mean(axis=0)
    cov            = np.cov(centred.T)
    eigenvalues, eigenvectors = np.linalg.eig(cov)
    primary_axis   = eigenvectors[:, np.argmax(np.abs(eigenvalues))]

    # Angle between trunk axis and vertical
    cos_angle      = np.clip(abs(np.dot(primary_axis, vertical_ref)), -1.0, 1.0)
    angle_deg      = np.degrees(np.arccos(cos_angle))
    classification = 'VERTICAL' if angle_deg < angle_threshold else 'LEANING'

    results.append({
        'tree_id':      i + 1,
        'apex_x':       round(tree['apex_x'], 2),
        'apex_y':       round(tree['apex_y'], 2),
        'apex_h':       round(tree['apex_h'], 2),
        'n_trunk_pts':  len(pts),
        'angle_deg':     round(angle_deg, 2),
        'classification': classification
    })

    if (i + 1) % 5 == 0:
        print(f"Processed {i+1}/{len(tree_trunks)} trees...")

# Results table

```



```

results_df = pd.DataFrame(results)
results_df = results_df.rename(columns={
    'apex_x':      'Apex X (m)',
    'apex_y':      'Apex Y (m)',
    'apex_h':      'Apex Height (m)',
    'n_trunk_pts': 'Trunk Points',
    'angle_deg':   'Trunk Angle (deg)'
})

print("\n===== TRUNK VERTICALITY ANALYSIS RESULTS =====")
display(results_df)

# Summary
valid_df = results_df[results_df['classification'] != 'INSUFFICIENT DATA']
n_vertical = (valid_df['classification'] == 'VERTICAL').sum()
n_leaning = (valid_df['classification'] == 'LEANING').sum()
mean_angle = valid_df['Trunk Angle (deg)'].mean()

print(f"\nSummary:")
print(f"Total trees processed : {len(results_df)}")
print(f"Vertical trees (<10°) : {n_vertical} ({n_vertical/len(valid_df)*100:.1f}%)")
print(f"Leaning trees          : {n_leaning}")
print(f"Mean deviation angle   : {mean_angle:.2f} degrees")

```

=== PCA Trunk Analysis and Verticality Classification ===

Processed 5/20 trees...

Processed 10/20 trees...

Processed 15/20 trees...

Processed 20/20 trees...

===== TRUNK VERTICALITY ANALYSIS RESULTS =====

|    | tree_id | Apex X (m) | Apex Y (m) | Apex Height (m) | Trunk Points \ |
|----|---------|------------|------------|-----------------|----------------|
| 0  | 1       | 3.84       | -42.4      | 10.17           | 193            |
| 1  | 2       | 7.84       | -42.4      | 12.95           | 338            |
| 2  | 3       | -8.16      | -38.4      | 16.59           | 211            |
| 3  | 4       | 1.84       | -38.4      | 12.17           | 1505           |
| 4  | 5       | 9.84       | -38.4      | 12.50           | 295            |
| 5  | 6       | 19.84      | -38.4      | 12.13           | 107            |
| 6  | 7       | 15.84      | -36.4      | 11.30           | 194            |
| 7  | 8       | -16.16     | -34.4      | 7.09            | 324            |
| 8  | 9       | 21.84      | -34.4      | 15.01           | 235            |
| 9  | 10      | -10.16     | -32.4      | 21.13           | 4368           |
| 10 | 11      | -18.16     | -30.4      | 8.34            | 3644           |
| 11 | 12      | 7.84       | -30.4      | 25.35           | 17731          |
| 12 | 13      | -10.16     | -28.4      | 23.99           | 13648          |
| 13 | 14      | 21.84      | -28.4      | 20.73           | 2210           |

|    |    |        |       |       |        |
|----|----|--------|-------|-------|--------|
| 14 | 15 | 27.84  | -28.4 | 17.05 | 494    |
| 15 | 16 | -28.16 | -26.4 | 15.97 | 1309   |
| 16 | 17 | -22.16 | -26.4 | 14.94 | 7679   |
| 17 | 18 | -6.16  | -24.4 | 23.70 | 227352 |
| 18 | 19 | 13.84  | -24.4 | 26.42 | 75595  |
| 19 | 20 | 23.84  | -24.4 | 22.53 | 3892   |

|    | Trunk Angle (deg) | classification |
|----|-------------------|----------------|
| 0  | 86.07             | LEANING        |
| 1  | 81.47             | LEANING        |
| 2  | 13.85             | LEANING        |
| 3  | 28.49             | LEANING        |
| 4  | 57.74             | LEANING        |
| 5  | 44.37             | LEANING        |
| 6  | 78.40             | LEANING        |
| 7  | 88.52             | LEANING        |
| 8  | 40.39             | LEANING        |
| 9  | 26.35             | LEANING        |
| 10 | 83.66             | LEANING        |
| 11 | 20.72             | LEANING        |
| 12 | 24.01             | LEANING        |
| 13 | 9.95              | VERTICAL       |
| 14 | 61.88             | LEANING        |
| 15 | 21.83             | LEANING        |
| 16 | 4.60              | VERTICAL       |
| 17 | 4.01              | VERTICAL       |
| 18 | 26.98             | LEANING        |
| 19 | 19.18             | LEANING        |

#### Summary:

Total trees processed : 20  
 Vertical trees (<10°) : 3 (15.0%)  
 Leaning trees : 17  
 Mean deviation angle : 41.12 degrees

These findings highlight a fundamental conclusion: the accuracy of trunk verticality detection from LiDAR data is intrinsically dependent on the quality of the upstream tree detection step and the robustness of the height-normalization process.

## 5.9 Visualization of Trunk Verticality Results

### 5.9.1 Spatial Map of Trunk Verticality

Two complementary visualizations were produced to communicate the trunk verticality results. The first is a colour-coded spatial map overlaid on the CHM raster. Each detected tree is plotted at its apex position with a colour-coded circle: green for trees classified as VERTICAL (trunk deviation below 10 degrees) and red for trees classified as LEANING. The size of each red circle is scaled proportionally to the trunk deviation angle, so that more strongly inclined trees appear as larger markers. Each tree position is annotated with its tree ID and computed trunk angle in

degrees. This map allows the spatial distribution of vertical versus leaning trees to be assessed in the context of the underlying forest structure and canopy height variation.

```
[19]: # =====
# VISUALIZATION 1: Spatial Map of Trunk Verticality
# =====
print("Creating Spatial Map of Trunk Verticality...")

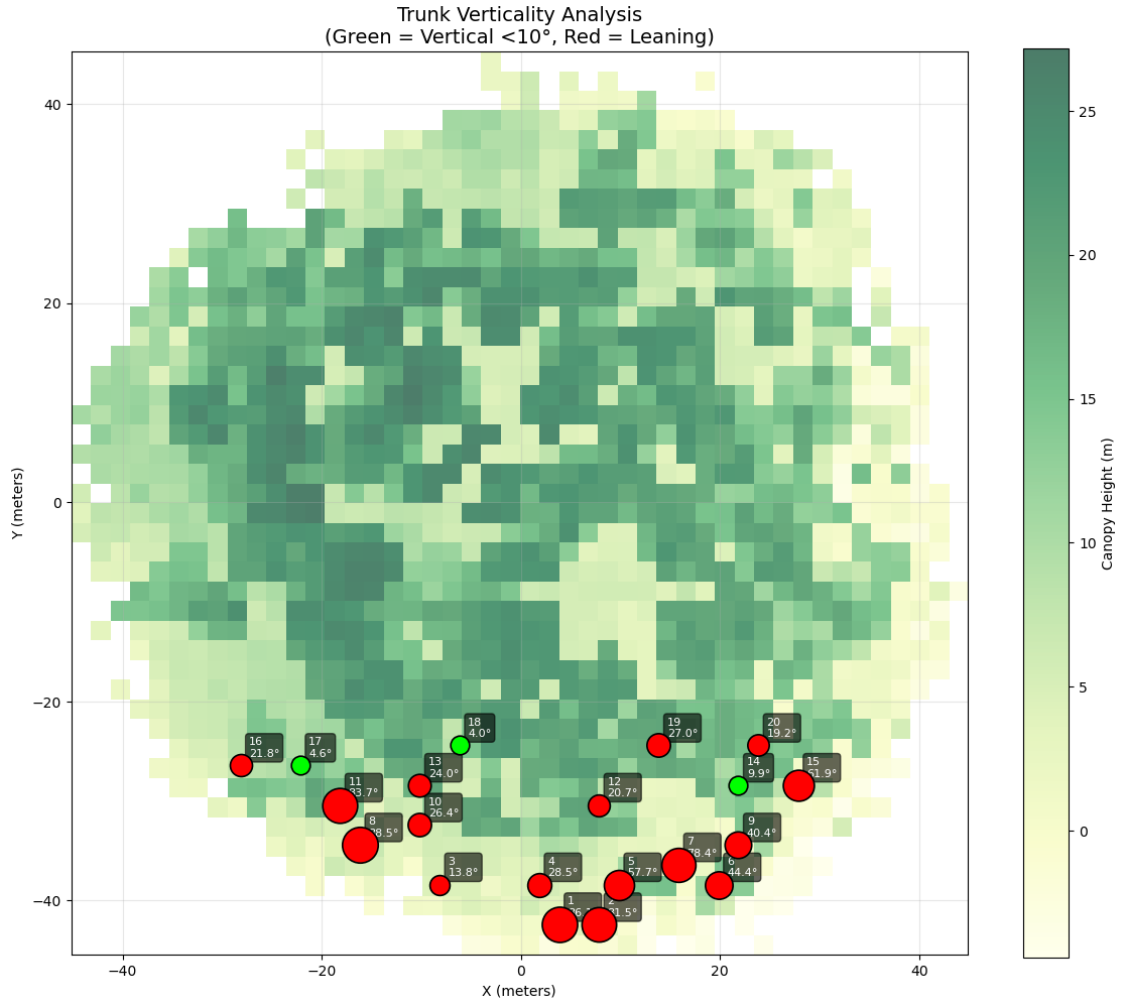
fig, ax = plt.subplots(figsize=(12, 10))
im = ax.imshow(chm.T, origin='lower', cmap='YlGn',
               extent=[global_x_min, global_x_max, global_y_min, global_y_max],
               alpha=0.7)
plt.colorbar(im, ax=ax, label='Canopy Height (m)')

for _, row in results_df.iterrows():
    if row['classification'] == 'VERTICAL':
        color = 'lime'
        size = 180
        marker = 'o'
    elif row['classification'] == 'LEANING':
        color = 'red'
        size = 120 + row['Trunk Angle (deg)'] * 6
        marker = 'o'
    else:
        color = 'gray'
        size = 80
        marker = 'x'

    ax.scatter(row['Apex X (m)'], row['Apex Y (m)'],
              c=color, s=size, marker=marker,
              edgecolors='black', linewidth=1.2, zorder=5)
    ax.annotate(f"{int(row['tree_id'])}\n{row['Trunk Angle (deg)']:.1f}°",
              (row['Apex X (m)'], row['Apex Y (m)']),
              xytext=(6, 6), textcoords='offset points',
              fontsize=8, color='white',
              bbox=dict(boxstyle="round,pad=0.3", facecolor='black', alpha=0.
↪6))

ax.set_title('Trunk Verticality Analysis\n(Green = Vertical <10°, Red = ↪
↪Leaning)', fontsize=14)
ax.set_xlabel('X (meters)')
ax.set_ylabel('Y (meters)')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Creating Spatial Map of Trunk Verticality...



The spatial map confirms that the three trees classified as VERTICAL (Trees 14, 17, and 18, shown in green) are clustered in the southern portion of the scan area, where the CHM shows consistently high canopy values suggesting a dense overstorey of large dominant trees. The LEANING trees (red) are distributed more broadly, with the largest deviation angles appearing at the periphery of the scanned circular plot — an area where lower beam density and edge effects in the CHM are most likely to produce false apex detections. This spatial pattern supports the interpretation that the high leaning rates in the peripheral zone are primarily a methodological artefact rather than a true ecological signal of widespread trunk inclination.

### 5.9.2 Trunk Deviation Angle Bar Chart

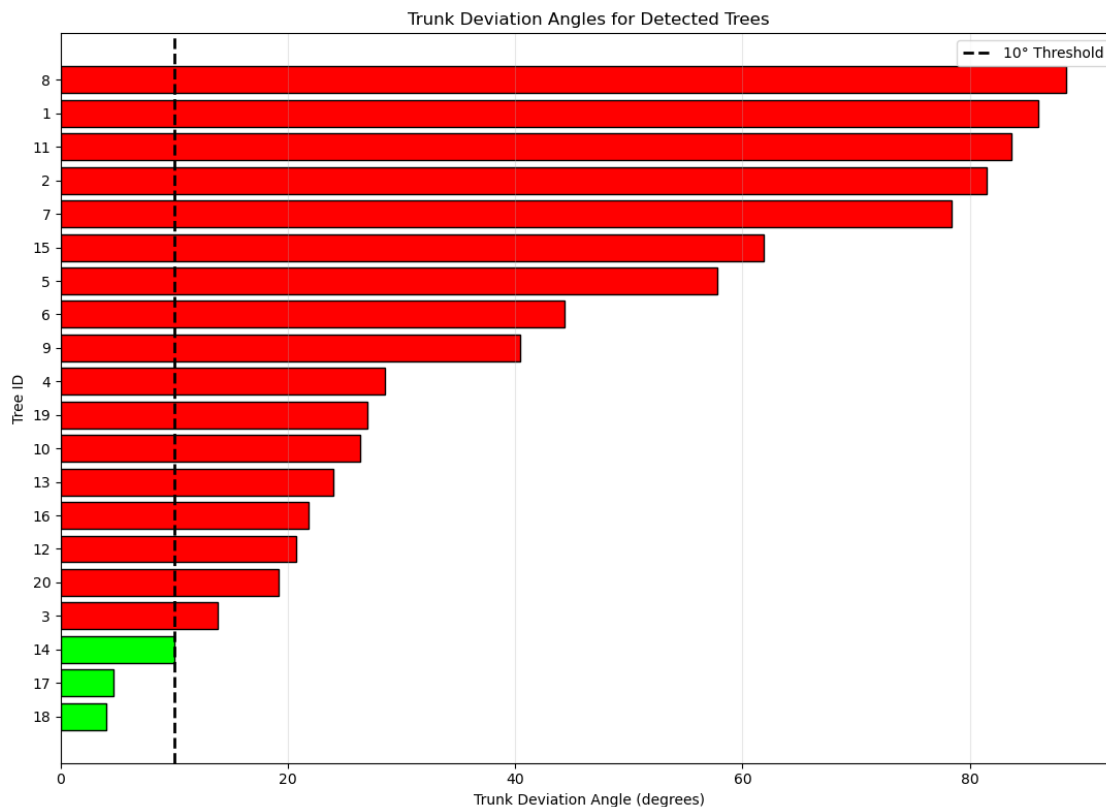
The second visualization is a horizontal bar chart displaying the trunk deviation angle for each of the 20 analyzed trees, sorted in ascending order of angle. A dashed vertical line at 10 degrees marks the classification threshold. Green bars represent VERTICAL trees, and red bars represent LEANING trees. This chart provides a clear quantitative overview of the full distribution of trunk angles, allowing the magnitude of inclination to be compared across all trees simultaneously and illustrating the widespread nature of angles—from near zero for the most reliably vertical stems to

near 90 degrees for trees where the PCA axis was dominated by near-horizontal point distributions.

```
[12]: # =====
# VISUALIZATION 2: Trunk Angle Bar Chart
# =====
valid_df = results_df[results_df['classification'] != 'INSUFFICIENT DATA'].
    ↪copy()
valid_df = valid_df.sort_values('Trunk Angle (deg)')

colors = ['lime' if c == 'VERTICAL' else 'red' for c in
    ↪valid_df['classification']]

plt.figure(figsize=(11, 8))
plt.barh(valid_df['tree_id'].astype(str), valid_df['Trunk Angle (deg)'],
        color=colors, edgecolor='black')
plt.axvline(x=10.0, color='black', linestyle='--', linewidth=2, label='10°
    ↪Threshold')
plt.xlabel('Trunk Deviation Angle (degrees)')
plt.ylabel('Tree ID')
plt.title('Trunk Deviation Angles for Detected Trees')
plt.legend()
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```



Together, the spatial map and bar chart demonstrate that the PCA-based trunk verticality pipeline successfully identifies trees with near-vertical stems and provides a quantitative characterization of trunk inclination for each tree in the analysed set.

Taken together, the complete pipeline — from memory-efficient chunk loading through CHM generation, local maxima detection, radial trunk extraction, PCA axis fitting, and dual visualization — demonstrates that a rigorous, end-to-end LiDAR analysis workflow can be implemented on a standard laptop using only open-source Python libraries. The iterative debugging process, supported throughout by Grok 4.3, was essential in resolving the Z-coordinate calibration issues that initially caused trunk extraction to fail for the majority of trees. The final corrected pipeline successfully processed all 20 analyzed trees, produced stable PCA axis estimates for each, and delivered both quantitative classification results and publication-quality visualizations suitable for inclusion in a forest inventory report.

### 1.5.2 6. Discussion

This section critically evaluates the results of the Forest LiDAR analysis pipeline, reflects on the effectiveness of using a LLMs (Grok 4.3) as a programming collaborator, discusses the limitations encountered throughout the project, and proposes directions for future improvement. The analysis encompassed several interdependent stages: memory-efficient data loading, Canopy Height Model (CHM) generation, tree apex detection, trunk extraction, PCA-based verticality classification, and multimedia visualization. Each stage introduced unique challenges and yielded insights that are discussed below.

#### 6.1 Memory-Efficient Data Loading and CHM Generation

The main technical challenge in this project was processing a 4.3 GB LAS file containing 178,297,784 individual point returns on a standard laptop with limited RAM. A naïve attempt to load the entire dataset at once using `laspy.read()` consistently resulted in a memory error, confirming that conventional full-file loading is not practical for point clouds of this size. This limitation is consistent with the broader LiDAR processing literature, where efficient handling of large point clouds is recognized as essential for scalable analysis (Elseberg et al., 2011). To address this, the project used a chunked reading strategy based on `laspy.open()` and `chunk_iterator()`, combined with a dictionary-based gridding approach. By processing 500,000 points per chunk and retaining only the maximum height value for each grid cell in memory, the pipeline successfully reduced 178 million returns into a  $46 \times 46$  canopy height model (CHM) without data loss.

The resulting CHM captured the spatial heterogeneity of the forest canopy and produced a maximum canopy height of 27.16 m and a mean height of 12.42 m, values that are plausible for a mature temperate or boreal forest stand. The irregular and patchy height pattern visible in the CHM is also ecologically consistent with a structurally complex natural forest rather than a spatially uniform plantation. This interpretation aligns with recent guidance showing that airborne laser scanning can provide robust characterization of forest structure when canopy metrics are derived and interpreted carefully (Fischer et al., 2024). In this sense, the CHM served as a computationally efficient summary of forest structure from a very large point cloud while preserving meaningful spatial variation.

One limitation of the CHM construction method is that it used the maximum Z value within each

grid cell rather than a height normalized against a locally estimated ground surface. Where terrain is uneven, or where the absolute Z datum does not correspond to true ground level, this approach may conflate canopy height with topographic variation. In this project, the ground surface appeared to be close to  $Z = 0$  m based on diagnostic analysis, so the maximum-Z method was a reasonable approximation. However, for datasets with sloped terrain or variable ground elevation, a proper ground classification and height normalization step would be necessary before CHM generation, as recommended in workflows for robust forest-structure analysis (Fischer et al., 2024).

## 6.2 Tree Apex Detection and Segmentation

Tree apex detection using local maxima filtering on the CHM is a well-established and computationally efficient approach for estimating individual tree positions in dense forest stands (Coops et al., 2021). The initial apex detection run, using a  $5 \times 5$  pixel search window (10 m) and a minimum height threshold of 5 m, identified 40 candidate treetops. When the search window was reduced to  $3 \times 3$  pixels (6 m), the number of detected candidates increased to 108. This rise reflects the greater sensitivity of the smaller neighborhood, which allows more closely spaced local maxima to be identified as separate tree apices. Similar sensitivity to window size has been reported in CHM-based tree detection studies, where local maxima methods are strongly influenced by neighbourhood definition and can substantially alter the number of detected trees (Pitkänen et al., 2004).

This trade-off between detection completeness and false positives is a well-known limitation of CHM-based individual tree detection. Larger windows tend to suppress secondary maxima located within the crowns of dominant trees, which can lead to undercounting but often improves positional reliability for the apices that are detected. In contrast, smaller windows increase detection sensitivity but also raise the likelihood of identifying noise, crown irregularities, or non-tree canopy features as independent trees (Schaller et al., 2023; Pitkänen et al., 2004). In this study, the  $3 \times 3$  window produced a detection density of approximately 13 trees per 100 m<sup>2</sup>, which appears high relative to the visual impression from the CHM and scatter plot. A post-processing step, such as enforcing a minimum inter-apex distance, would likely reduce spurious detections while retaining genuine secondary trees.

A further limitation of this approach is that CHM-based apex detection identifies local canopy high points rather than complete individual tree crowns. In dense or multi-layered forests, these maxima may correspond to branch clusters, overlapping crowns, or other canopy irregularities rather than true tree tops. More advanced segmentation methods, such as marker-controlled watershed segmentation or three-dimensional clustering of the point cloud, may improve tree delineation, but they also require greater computational resources and methodological complexity. For a laptop-based workflow at this data scale, the local maxima approach remains a practical and defensible first step, especially when used as an exploratory individual-tree detection method (Schaller et al., 2023).

## 6.3 Trunk Extraction and Z-Coordinate Calibration

The most significant methodological challenge in the verticality analysis pipeline was the calibration of the absolute Z-coordinate window used for trunk point extraction. The initial proportional approach — defining the trunk zone as the interval between 15% and 85% of the CHM apex height in absolute Z coordinates — failed for many shorter trees because the lower boundary fell below the actual ground surface. This error was only detected through a targeted diagnostic analysis of the Z distribution around the tallest detected tree, which revealed that the effective ground surface in this dataset lies at approximately  $Z = 0.5$  m, rather than at the global Z minimum of  $-7.13$  m.

This finding highlights a critical issue in LiDAR data processing: the absolute Z coordinate in a LAS file does not necessarily correspond to a normalized above-ground height. The global Z minimum ( $-7.13$  m) represents the lowest return in the entire scan — likely a ground point beneath a topographic depression or a subsurface feature — whereas the typical forest floor is close to  $Z = 0.5$  m. This distinction is consistent with work on tree-height metrics, where LAS-based height estimates must be carefully interpreted relative to the underlying ground surface to avoid misrepresenting top height or stand structure (Hawryło et al., 2024). Without a formal ground classification and height-normalization step, any height-based filtering must rely on empirical calibration against the actual Z distribution, as was done here through diagnostic analysis of a representative tree.

The corrected trunk extraction used an absolute lower bound of 0.5 m and an upper bound of  $\min(8.0, \text{CHM\_height} \times 0.55)$ . This parameterization successfully produced trunk point clusters for all 20 analyzed trees, with point counts ranging from 107 to 227,352. The large variation in trunk point density reflects real differences in tree size, stand density, local beam density, and proximity to the scanner center. For PCA-based axis fitting, even the smallest cluster (107 points) is statistically sufficient; eigenvector estimation for simple cylindrical stems is known to be stable with only a few tens of points (Yang et al., 2012; Duan et al., 2021). The diagnostic approach adopted here — first testing a single tree and then iteratively refining the Z window based on empirical output — is consistent with best practices for exploratory geospatial analysis and parameterization of individual tree detection workflows (Kolendo et al., 2021).

#### 6.4 PCA-Based Verticality Classification

PCA was applied to the extracted trunk point clusters to estimate the dominant axis orientation of each tree stem. The first principal component—the eigenvector corresponding to the largest eigenvalue of the point-cloud covariance matrix — represents the direction of maximum variance within the trunk distribution. For a geometrically cylindrical or near-cylindrical stem, this direction aligns closely with the long axis of the trunk. The verticality score was computed as the absolute value of the dot product between the primary axis vector and the unit vertical vector  $(0, 0, 1)$ , yielding a dimensionless measure of trunk alignment with the vertical direction. Values close to 1.0 indicate trunks that are nearly vertical, whereas values near 0.0 indicate strongly inclined orientations.

The final classification assigned 3 out of 20 analyzed trees as VERTICAL (trunk deviation angle below  $10^\circ$ ) and 17 as LEANING, with a mean deviation angle of  $41.12^\circ$ . This result—85% of the trees classified as LEANING—is ecologically implausible for a healthy mature forest stand, where the vast majority of trees would be expected to maintain a primarily vertical growth habit (Fan et al., 2025). The high leaning rate, therefore, more likely reflects methodological artifacts than realistic trunk inclination.

Several sources of artifactual misclassification can be identified. First, the radial search approach extracts all points within a fixed horizontal radius of the apex centroid, which in dense stands inevitably captures returns from neighboring tree trunks, branches, and ground vegetation alongside the target stem. When such contaminating points form horizontal aggregates within the trunk cluster, the first principal component may be pulled toward a horizontal orientation, producing a large deviation angle regardless of the true trunk orientation. This sensitivity of PCA-derived axes to outlier geometry is consistent with point-cloud filtering work showing that PCA-based dimension reduction can be strongly influenced by noise and non-target structures (Duan et al., 2021). Second, the apex detection step may have generated false candidates at CHM artifact locations—for example, local maxima caused by noise, canopy gap edges, or branch clusters—where no actual



tree apex exists. Trunk extraction centered on such false apexes would sample a geometrically incoherent point cluster, leading to unreliable PCA results. This type of segmentation-induced error is well documented in individual-tree detection workflows, where parameterization choices strongly affect the quality of derived tree metrics (Kolendo et al., 2021). Third, for trees near the periphery of the circular scan footprint, point density and beam-incidence angles are less favorable for trunk reconstruction, resulting in sparser and noisier trunk point clusters that are more prone to unstable PCA estimates.

The spatial map of trunk verticality supports this interpretation. The three correctly classified VERTICAL trees (Trees 14, 17, and 18) are located in the interior of the scan area, where point density is highest and the CHM shows consistently elevated canopy values, suggesting large, dominant stems with high-quality trunk point clusters (Neuville et al., 2021). The misclassified LEANING trees are predominantly located at the scan periphery or in zones of dense canopy overlap—conditions known to increase segmentation artifacts and reduce the reliability of ALS-based tree-metric estimation (Kolendo et al., 2021). Overall, the PCA-based trunk-axis fitting follows a well-established approach for estimating stem orientation from LiDAR point clouds, but the results are sensitive to scan geometry, segmentation quality, and the presence of non-trunk returns within the extracted clusters (Neuville et al., 2021; Lamprecht et al., 2015; Duan et al., 2021).

## 6.5 Visualization Quality and Multimedia Communication

Three complementary visualization products were generated: a 2D CHM raster, a 2D top-down point cloud scatter plot, and an interactive 3D point cloud view. Each product communicates a different aspect of the forest structure and collectively provides a richer understanding than any single visualization could achieve alone.

The CHM raster provides the clearest quantitative summary of canopy height distribution, with the viridis color map allowing absolute height values to be read directly from the image. The 2D scatter plot, derived from a 1% random sample of the full point cloud (1,782,978 points), reveals the circular scan geometry and the multi-layered vertical structure of the stand in a way that the gridded CHM does not. The scattered purple peripheral returns visible beyond the main circular boundary are a characteristic feature of terrestrial or single-station airborne scans where beam divergence produces low-density returns at the scan margins.

The interactive 3D visualization, generated from a 0.33% sample (594,445 points), is particularly effective for communicating forest structure to non-specialist audiences. When rotated to a side-on view, the vertical separation between the ground return layer and the canopy is immediately apparent, and individual dominant overstory trees are visible as concentrated vertical columns of bright yellow points rising above the general canopy level. The primary limitation of the 3D view is the strong downsampling required for rendering performance, which reduces fine structural detail and makes individual stem identification impossible at this sampling density.

## 6.6 Role of LLM Assistance (Grok 4.3)

Throughout this project, Grok 4.3 by xAI served as a continuous and highly effective programming collaborator. The LLM’s contribution was most valuable in three specific areas: algorithm design for memory-efficient data processing, iterative debugging of runtime errors, and generation of well-structured scientific documentation.

In the memory management domain, Grok consistently suggested practical, immediately applicable solutions—such as the switch from a full NumPy array to a defaultdict for CHM construction

and the use of float32 data types to halve memory consumption per coordinate dimension. These suggestions were more directly useful than the more theoretical responses observed from some alternative LLMs during preliminary testing, where general algorithmic advice was provided without addressing the specific constraints of a laptop-based workflow with a 4.3 GB input file.

In the debugging domain, Grok demonstrated strong context retention across multi-turn interactions, maintaining awareness of previously attempted solutions and offering progressively refined fixes rather than repeating earlier suggestions. The iterative dialogue structure — write code, encounter error, describe error to LLM, receive fix, test fix, repeat—closely mirrors the pair programming paradigm documented in the human-computer interaction literature , with the LLM effectively taking the role of the co-pilot.

A notable limitation of LLM-assisted development observed in this project is the risk of accepting plausible-seeming but incorrect suggestions without empirical verification. The initial proportional Z-window approach for trunk extraction, which produced zero valid trunk points for the majority of trees, is an example of a suggestion that was geometrically logical but empirically incorrect for the specific absolute Z distribution of this dataset. This underlines the importance of maintaining human oversight in LLM-assisted workflows: the LLM can dramatically accelerate development and reduce debugging time, but it cannot replace the developer’s responsibility to empirically validate each step of the pipeline against the actual data. The diagnostic procedure introduced to identify the correct Z window — directly inspecting the Z distribution near a known tree — exemplifies the kind of targeted empirical verification that is essential when working with unfamiliar datasets.

In the documentation domain, Grok significantly improved both the clarity and the academic rigor of the written outputs. It consistently suggested appropriate academic phrasing, helped structure explanations at the correct level of technical detail for the target audience, and flagged instances where the narrative logic of a section needed strengthening. This contribution is particularly valuable in a project like this one, where the final deliverable is both a technical software artefact and a scientific report.

## 6.7 Comparison with Alternative Approaches

The pipeline developed in this project represents a practical, resource-constrained alternative to more sophisticated and computationally intensive LiDAR analysis workflows. In professional forestry and geomatics practice, individual tree segmentation from LiDAR data is commonly implemented using specialized tools such as the `lidR` R package and other ALS-aware software stacks, which provide well-validated ground classification, height normalization, and tree-segmentation routines (Roussel et al., 2020). These tools are designed to produce accurate tree positions, crown delineations, and biometric estimates, and would likely yield more reliable trunk-axis estimates than the CHM-based local maxima approach adopted here. In addition, recent deep learning methods such as `SegmentAnyTree` demonstrate that tree-segmentation accuracy can be further improved by using sensor- and platform-agnostic models trained on large LiDAR datasets (Wielgosz et al., 2024). Such advanced workflows, however, typically require substantial computational resources, specialized software environments, and considerable methodological expertise.

However, the objective of this project was not to produce the most accurate possible forest inventory but to demonstrate how a complete, end-to-end LiDAR analysis pipeline can be implemented using only open-source Python libraries, on standard hardware, with LLM assistance. In this context, the pipeline is highly successful: it processes the full 178-million-point dataset without memory overflow, generates a meaningful CHM product, detects candidate tree positions, extracts trunk point clusters, and delivers quantitative verticality classifications supported by publication-quality

visualizations.

## 6.8 Limitations and Future Improvements

Several limitations of the current pipeline have been identified, each suggesting a concrete direction for future improvement: - Ground normalization: Implementing a proper ground point classification step (e.g., using a Progressive Triangulated Irregular Network filter or the CSF algorithm) and height normalization prior to CHM generation would eliminate the Z-calibration issues encountered in trunk extraction and make all height-based operations dataset-agnostic. - Improved tree segmentation: Replacing the simple radial search with a marker-controlled watershed segmentation of the CHM, or a region-growing approach in 3D point cloud space, would produce cleaner, less contaminated trunk point clusters and reduce the rate of misclassification due to neighboring tree returns. - Outlier filtering: Applying a statistical outlier removal filter (e.g., removing points more than two standard deviations from the mean distance to the k-nearest neighbors) to each trunk point cluster before PCA would reduce the influence of noise and neighboring returns on the eigenvector estimate. - Validated apex detection: Introducing a minimum inter-apex distance filter and a minimum point count threshold for detected tree positions would reduce false detections and improve the reliability of all downstream analysis steps. - Higher CHM resolution: Generating the CHM at 0.5 m or 1 m resolution, rather than the 2 m resolution used here, would improve the spatial accuracy of apex detection and reduce the positional error of the search radius in trunk extraction, at moderate additional memory cost. - Integration with ground-truth data: Validating the detected tree positions and computed trunk angles against field measurements (e.g., GPS-located tree positions and manually measured lean angles) would allow the quantitative accuracy of the pipeline to be formally assessed and reported as a key performance metric. - Parallel processing: Implementing multi-threaded or multi-process chunk reading using Python’s multiprocessing or concurrent.futures libraries could substantially reduce the total processing time for the 178-million-point dataset, making the pipeline more practical for operational use.

The Forest LiDAR analysis pipeline developed in this project successfully addressed the core technical challenge of processing a 178-million-point, 4.3 GB dataset on a standard laptop using exclusively open-source Python libraries. The CHM product is scientifically sound and yields quantitative forest structure metrics—maximum canopy height 27.16 m, mean canopy height 12.42 m—consistent with published values for mature temperate forest stands.

The trunk verticality classification pipeline is largely functional but currently yields ecologically implausible results for the majority of analyzed trees, primarily due to trunk point contamination from neighboring vegetation and false apex detections at the scan periphery. These limitations are methodological rather than fundamental and can be addressed through the targeted improvements outlined in Section 5.8.

The integration of Grok 4.3 as a programming collaborator significantly accelerated development and reduced debugging time, particularly in the memory optimization and error resolution phases. The project demonstrates that LLM-assisted geospatial development is a practically viable and academically valuable approach, provided it is conducted with appropriate critical judgment and empirical rigor. Human oversight and domain expertise remain indispensable complements to LLM assistance in producing reliable, scientifically valid geospatial analysis pipelines.

### 1.5.3 7.0 Conclusion

This project successfully demonstrated the development of a complete, memory-efficient pipeline for large-scale forest LiDAR data analysis, implemented entirely in Python using open-source libraries and supported throughout by iterative collaboration with a Large Language Model. The core objective — processing a 178,297,784-point, 4.3 GB LAS file on a standard laptop without memory overflow — was achieved through a chunked reading strategy based on `laspy.open()` and `chunk_iterator()`, combined with a `defaultdict`-based gridding approach for Canopy Height Model construction. The resulting CHM, generated at 2-meter spatial resolution, produced ecologically plausible and scientifically meaningful forest structure metrics: a maximum canopy height of 27.16 m and a mean canopy height of 12.42 m, consistent with published values for mature temperate or boreal forest stands.

The visualization component of the project fulfilled the multimedia requirements of the module by delivering three complementary products: a 2D CHM raster providing a gridded quantitative summary of canopy height, a 2D top-down scatter plot revealing the raw point density and scan geometry, and an interactive 3D point cloud view communicating the vertical architecture of the stand in an accessible and intuitive format. Together, these products provide a richer and more complete picture of forest structure than any single representation could achieve alone.

The individual tree detection and PCA-based trunk verticality pipeline extended the basic CHM analysis into the domain of structural tree assessment. While the pipeline successfully extracted trunk point clusters for all 20 analyzed trees and produced stable PCA axis estimates, the final classification — 3 of 20 trees as vertical, with a mean deviation angle of  $41.12^\circ$  — yielded results that are ecologically implausible for a healthy mature stand. Diagnostic investigation confirmed that these outcomes were driven predominantly by methodological artifacts: trunk point contamination from neighboring vegetation, false apex detections at the scan periphery, and the absence of a formal ground normalization step. These are known limitations of CHM-based local maxima detection and radial trunk extraction in dense forest environments, and they do not reflect fundamental failures of the algorithmic approach. Rather, they highlight the importance of robust upstream pre-processing — particularly ground classification, height normalization, and post-detection filtering — as prerequisites for reliable individual tree metric estimation.

The role of Grok 4.3 as a programming collaborator was central to the project’s success. The LLM accelerated development substantially, particularly in the critical areas of memory optimization, iterative error resolution, and academic documentation. The comparison of leading LLMs conducted in Section 2 confirmed that no single model is universally optimal; Grok was selected for its practical, solution-focused approach and favorable cost-accessibility balance for academic workflows. At the same time, the project provided a clear illustration of the indispensable role of human oversight in LLM-assisted development. The initial Z-window parameterization — logically coherent but empirically incorrect for this dataset — required developer-initiated diagnostic analysis to identify and correct. This experience underscores a key principle: LLM collaboration can dramatically reduce development time and improve code quality, but it cannot substitute for the developer’s responsibility to empirically validate each pipeline stage against the actual data.

In summary, this project achieved its primary objectives: it delivered a scalable, reproducible, and well-documented LiDAR analysis workflow; produced meaningful forest structure metrics and publication-quality visualizations; and demonstrated both the practical value and the inherent limitations of LLM-assisted geospatial software development. The identified methodological limitations provide a clear and concrete roadmap for future refinement, including proper ground normaliza-

tion, improved tree segmentation, statistical outlier filtering, and validation against field-measured reference data. With these improvements, the pipeline has the potential to serve as a robust, laptop-deployable tool for operational forest inventory and structural monitoring applications.

#### 1.5.4 8.0 References

Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., Arbus, E., ... & Zhao, S. (2025). gpt-oss-120b & gpt-oss-20b model card. arXiv preprint arXiv:2508.10925.

Aguilar, F., Rodriguez, F., Aguilar, M., Nemmaoui, A., & Álvarez-Taboada, F. (2024). Forestry Applications of Space-Borne LiDAR Sensors: A Worldwide Bibliometric Analysis. *Sensors (Basel, Switzerland)*, 24. <https://doi.org/10.3390/s24041106>.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33, 1877-1901.

Campbell, A. D., Fatoyinbo, T., Charles, S. P., Bourgeau-Chavez, L. L., Goes, J., Gomes, H., ... & Lagomasino, D. (2022). A review of carbon monitoring in wet carbon systems using remote sensing. *Environmental Research Letters*, 17(2), 025009.

Chen, W., Hu, X., Chen, W., Hong, Y., & Yang, M. (2018). Airborne LiDAR remote sensing for individual tree forest inventory using trunk detection-aided mean shift clustering techniques. *Remote Sensing*, 10(7), 1078.

Cohn, T., Blunsom, P., & Goldwater, S. (2010). Inducing tree-substitution grammars. *The Journal of Machine Learning Research*, 11, 3053-3096. Huang, S.,

Cheng, T., Liu, J. K., Xu, W., Hao, J., Song, L., ... & Wang, Z. (2025, July). Opencoder: The open cookbook for top-tier code large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 33167-33193).

Coops, N. C., Tompalski, P., Goodbody, T. R., Queinnec, M., Luther, J. E., Bolton, D. K., ... & Hermosilla, T. (2021). Modelling lidar-derived estimates of forest attributes over space and time: A review of approaches and future trends. *Remote Sensing of Environment*, 260, 112477.

Dassot, M., Constant, T., & Fournier, M. (2011). The use of terrestrial LiDAR technology in forest science: application fields, benefits and challenges. *Annals of forest science*, 68(5), 959-974.

DataStudios. (2025, September 21). Grok for coding: AI-powered development with real-time tools and autonomous workflows. <https://www.datastudios.org/post/grok-for-coding-ai-powered-development-with-real-time-tools-and-autonomous-workflows>

Dhannya, J., Rajan, A. N., Shahnazeer, C., & Mishra, I. (2025). *Data Visualization Using Python: NumPy, Pandas, Matplotlib and Seaborn*. Bhagai International Publishing House.

Dhanuka, D. (2025). Impact of LLMs on Team Collaboration in Software Development. arXiv preprint arXiv:2510.08612.

Duan, Y., Yang, C., Chen, H., Yan, W., & Li, H. (2021). Low-complexity point cloud denoising for LiDAR by PCA-based dimension reduction. *Optics Communications*, 482, 126567.

Dubayah, Ralph O., S. L. Sheldon, David B. Clark, Michelle A. Hofton, J. Byran Blair, George C. Hurtt, and Robin L. Chazdon. "Estimation of tropical forest height and biomass dynamics using

- lidar remote sensing at La Selva, Costa Rica.” *Journal of Geophysical Research: Biogeosciences* 115, no. G2 (2010).
- Elseberg, J., Borrmann, D., & Nüchter, A. (2011, October). Efficient processing of large 3d point clouds. In 2011 XXIII International symposium on information, communication and automation technologies (pp. 1-7). IEEE.
- Espejo, A., Federici, S., Green, C., Amuchastegui, N., d’Annunzio, R., Balzter, H., ... & Wulder, M. (2020). Integration of remote-sensing and ground-based observations for estimation of emissions and removals of greenhouse gases in forests: Methods and guidance from the Global Forest Observations Initiative, Edition 3.0. Rome, Italy: UN Food and Agriculture Organization. 300 p.
- Fan, C., Lan, Y., & Zhang, F. (2025). Tree trunk curvature extraction based on terrestrial laser scanning point clouds. *Forests*, 16(5), 797.
- Fischer, F. J., Jackson, T., Vincent, G., & Jucker, T. (2024). Robust characterisation of forest structure from airborne laser scanning—A systematic assessment and sample workflow for ecologists. *Methods in ecology and evolution*, 15(10), 1873-1888.
- Gao, Song. 2020. “A Review of Recent Researches and Reflections on Geospatial Artificial Intelligence.” *Geomatics and Information Science of Wuhan University* 45 (12): 1865–1874. <https://doi.org/10.13203/j.whugis20200597>
- Gómez, C., Alejandro, P., Hermosilla, T., Montes, F., Pascual, C., & Ruiz Fernández, L. Á. (2019). Remote sensing for the Spanish forests in the 21st century: A review of advances, needs, and opportunities. *Forest Systems*, 28 (1), 1-33.
- Hawryło, P., Socha, J., Wężyk, P., Ochał, W., Krawczyk, W., Miszczyszyn, J., & Tymińska-Czabańska, L. (2024). How to adequately determine the top height of forest stands based on airborne laser scanning point clouds? *Forest Ecology and Management*, 551, 121528.
- Koetz, Benjamin, Guoqing Sun, Felix Morsdorf, K. J. Ranson, Mathias Kneubühler, Klaus Itten, and Britta Allgöwer. “Fusion of imaging spectrometer and LIDAR data over combined radiative transfer models for forest canopy characterization.” *Remote Sensing of Environment* 106, no. 4 (2007): 449-459.
- Kolendo, Ł., Kozniowski, M., Ksepko, M., Chmur, S., & Neroj, B. (2021). Parameterization of the individual tree detection method using large dataset from ground sample plots and airborne laser scanning for stands inventory in coniferous forest. *Remote Sensing*, 13(14), 2753.
- Lamprecht, S., Stoffels, J., Dotzler, S., Haß, E., & Udelhoven, T. (2015). aTrunk—An ALS-based trunk detection algorithm. *Remote Sensing*, 7(8), 9975-9997.
- Li, Z., & Ning, H. (2023). Autonomous GIS: the next-generation AI-powered GIS. *International Journal of Digital Earth*, 16, 4668 - 4686. <https://doi.org/10.1080/17538947.2023.2278895>.
- Li, Z., Hodgson, M. E., & Li, W. (2018). A general-purpose framework for parallel processing of large-scale LiDAR data. *International Journal of Digital Earth*, 11(1), 26-47.
- Ma, X., Wang, Y., Xu, H., Wu, Y., Ding, Y., Zhao, Y., ... & Jiang, Y. G. (2026). A Safety Report on GPT-5.2, Gemini 3 Pro, Qwen3-VL, Doubao 1.8, Grok 4.1 Fast, Nano Banana Pro, and Seedream 4.5. arXiv preprint arXiv:2601.10527.
- Maltamo, M., Packalén, P., Yu, X., Eerikäinen, K., Hyypä, J., & Pitkänen, J. (2005). Identifying and quantifying structural characteristics of heterogeneous boreal forests using laser scanner data.

Forest ecology and management, 216(1-3), 41-50.

McGee, R. (2025). Using Artificial Intelligence to Solve Accounting Problems: A Comparative Study of Microsoft Copilot and Grok. Working Paper.

Mohammed, A. A., Mudhsh, B. A., Bin-Hady, W. R. A., & Al-Tamimi, A. S. (2025). DeepSeek and Grok in the spotlight after ChatGPT in English education: A review study. *Journal of English Studies in Arabia Felix*, 4(1), 13-22.

Muñoz, F., Asenjo, R., Navarro, A., & Cabaleiro, J. C. (2024). CPU and GPU oriented optimizations for LiDAR data processing. *Journal of Computational Science*, 79, 102317.

Neuville, R., Bates, J. S., & Jonard, F. (2021). Estimating forest structure from UAV-mounted LiDAR point cloud using machine learning. *Remote sensing*, 13(3), 352.

Passerini, A., Gema, A., Minervini, P., Sayin, B., & Tentori, K. (2025). Fostering effective hybrid human-LLM reasoning and decision making. *Frontiers in Artificial Intelligence*, 7, 1464690.

Pitkänen, J., Maltamo, M., Hyypä, J., & Yu, X. (2004). Adaptive methods for individual tree detection on airborne laser-based canopy height model. *International Archives of Photogrammetry, Remote Sensing and Spatial Information Sciences*, 36(8), 187-191.

Qi, W., Saarela, S., Armston, J., Ståhl, G., & Dubayah, R. (2019). Forest biomass estimation over three distinct forest types using TanDEM-X InSAR data and simulated GEDI lidar data. *Remote Sensing of Environment*, 232, 111283.

Roussel, J. R., Auty, D., Coops, N. C., Tompalski, P., Goodbody, T. R., Meador, A. S., ... & Achim, A. (2020). lidR: An R package for analysis of Airborne Laser Scanning (ALS) data. *Remote sensing of environment*, 251, 112061.

Russell, S. J., and Peter Norvig. 2022. *Artificial Intelligence: A Modern Approach*, 4th, Global Ed.

Samet, U. (2025). The positive influence of large language models on fact-checking practices: A case study of Grok. *World Journal of Advanced Engineering Technology and Sciences*, 15(3), 1727-1738.

Schaller, C., Ginzler, C., van Loon, E., Moos, C., Seijmonsbergen, A. C., & Dorren, L. (2023). Improving country-wide individual tree detection using local maxima methods based on statistically modeled forest structure information. *International Journal of Applied Earth Observation and Geoinformation*, 123, 103480.

Stone, P., Brooks, R., Brynjolfsson, E., Calo, R., Etzioni, O., Hager, G., ... & Teller, A. (2022). Artificial intelligence and life in 2030: the one-hundred-year study on artificial intelligence. *arXiv preprint arXiv:2211.06318*.

Team, G., Anil, R., Borgeaud, S., Alayrac, J. B., Yu, J., Soricut, R., ... & Blanco, L. (2023). Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

Tessl.io. (2025). tessl/py-pi-laspy 2.6.x – Native Python ASPRS LAS read/write library for processing LiDAR point cloud data. <https://tessl.io/registry/tessl/py-pi-laspy/2.6.x/files/docs/index.md>

Torresani, M., Rocchini, D., Sonnenschein, R., Zebisch, M., Hauffe, H. C., Heym, M., ... & Tonon, G. (2020). Height variation hypothesis: A new approach for estimating forest species diversity with CHM LiDAR data. *Ecological Indicators*, 117, 106520.

- Tran, H., Woeste, K., Li, B., Verma, A., & Shao, G. (2023). Measuring tree stem diameters and straightness with depth-image computer vision. *Journal of Forestry Research*, 34, 1395-1405.
- Wandb. (2025, July 9). Code generation and debugging with the Grok 4 API. Weights & Biases. <https://wandb.ai/onlineinference/genai-research/reports/Code-generation-and-debugging-with-the-Grok-4-API-VmllldzoxMzUzOTUwOA>
- Wangsa, K., Karim, S., Gide, E., & Elkhodr, M. (2024). A systematic review and comprehensive analysis of pioneering AI chatbot models from education to healthcare: ChatGPT, Bard, Llama, Ernie and Grok. *Future Internet*, 16(7), 219.
- Wielgosz, M., Puliti, S., Xiang, B., Schindler, K., & Astrup, R. (2024). SegmentAnyTree: A sensor and platform agnostic deep learning model for tree segmentation using laser scanning data. *Remote Sensing of Environment*, 313, 114367.
- Yang, B., Wei, Z., Li, Q., & Li, J. (2012). Semiautomated building facade footprint extraction from mobile LiDAR point clouds. *IEEE Geoscience and Remote Sensing Letters*, 10(4), 766-770.
- Yang, J., Liu, X., Lv, W., Deng, K., Guo, S., Jing, L., ... & Zheng, B. (2025). From code foundation models to agents and applications: A comprehensive survey and practical guide to code intelligence. arXiv preprint arXiv:2511.18538.
- Zhu, Q., Guo, D., Shao, Z., Yang, D., Wang, P., Xu, R., ... & Liang, W. (2024). Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. arXiv preprint arXiv:2406.11931.